



Hybridizing differential evolution and local search optimization for dimensional synthesis of linkages



Ramon Sancibrian^{a,*}, Angel Sedano^b, Esther G. Sarabia^c, Jesus M. Blanco^d

^a Department of Structural and Mechanical Engineering, University of Cantabria, Avda. de los Castros s/n, 39005 Santander, Spain

^b Asset Management Services: Renewables, UL, C/ Larragueta 8B, 310013 Ansoain, Navarra, Spain

^c Department of Electronic Technology and Automatic Systems, University of Cantabria, Avda. de los Castros s/n 39005 Santander, Spain

^d Department of Nuclear Engineering and Fluid Mechanics, University of the Basque Country (UPV/EHU), Plaza Ingeniero Torres Quevedo, Edif. 1, 48013 Bilbao, Spain

ARTICLE INFO

Article history:

Received 20 February 2019

Revised 17 May 2019

Accepted 12 June 2019

Available online 19 June 2019

Keywords:

Synthesis

Hybrid optimization

Evolutionary algorithm

Local search

ABSTRACT

This work deals with the optimal dimensional synthesis of planar linkages. The paper proposes a new search procedure to obtain the best linkage by means of hybridizing a Local Search (LS) approach and an Evolutionary Algorithm (EA). Hybrid Algorithms (HAs) combine the advantages of stochastic and deterministic optimization, enhancing their features in the search for the best solution. However, the way in which the integration of these two kinds of algorithms should be implemented is not a trivial issue and many options exist that have to be investigated. In this paper different schemes of hybrid algorithms are applied to the synthesis of planar linkages. The work discusses different approaches that use information and structures for achieving a balance between global and local searches. Differential Evolution (DE) is used together with the Generalized Reduced Gradient (GRG) approach for optimal search. Both Cluster and Elite Analyses are studied to select an effective hybrid optimization strategy. The different schemes of hybrid algorithms are presented and analyzed in different examples for the design of planar linkages and the best options are highlighted.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Dimensional synthesis deals with the determination of the dimensions of the links of a mechanism in such a way that the prescribed task can be performed properly. Dimensional synthesis has been widely studied in the literature in the form of three classical problems, which are defined as path generation, function generation and rigid-body guidance [1]. All these problems lead to highly nonlinear formulations [2], which are very difficult to solve even with the help of powerful computers. This difficulty arises when the number of links or prescribed positions is high, as happens in the design of complex mechanisms [3]. For this reason, in the last decades, most research has been focused on the development and adaptation of accurate, efficient and robust numerical algorithms to solve the synthesis problem [4]. The goal is to solve highly nonlinear problems, subject to a set of constraints that normally appear in this type of designs. Numerical optimization is one of the main techniques used successfully in these kinds of problems [5], as it facilitates the formulation of a prescribed task as a goal function, enabling the assessment of how good a mechanism is at satisfying the imposed requirements.

* Corresponding author.

E-mail address: sancibrr@unican.es (R. Sancibrian).

The first works performing optimization in the design of planar mechanisms appeared in the seventies [6–9]. These works used deterministic LS techniques to find the closest minimum from an initial guess linkage. Since then, many improvements have been made to the basic local search optimization approaches. One of the main drawbacks in the first optimization methods was the use of an approximate gradient. Thus, the estimated direction of descent lacked the necessary accuracy, causing a loss of efficiency and robustness [10,11]. Exact gradient determination was one of the main features that provided an important enhancement to these approaches. For instance, by using an exact gradient, Mariappan et al. [12] improved the accuracy of these techniques in determining the direction of descent. A generalized exact-gradient based optimization was also proposed by Sancibrian et al. [13,14] for the three classical problems in dimensional synthesis. However, generalized exact-gradient determination provides the necessary information to reach the minimum only if the goal function is well formulated. Indeed, the way in which the mechanism's error is formulated plays a crucial role in optimization. The goal function is subject to a set of constraints that must include the main requirements of the linkage. The typical formulation of this goal function measures the difference between the desired and generated path by using the Euclidean distance. This concept of synthesis error was widely used in the first approaches, having important strengths but also several limitations [15]. Ullah and Kota [16], who proposed Fourier Descriptors as a tool to evaluate the shape differences, presented one of the first works improving the formulation of the goal function. However, the use of Fourier Descriptors is limited to closed paths, which constitutes a strong limitation in dimensional synthesis. To overcome this problem, wavelets were proposed by Galan et al. [17] to determine small changes in the shape of the curves. Sedano et al. [15] introduced the concept of error estimator including three steps in the measurement of the error. These steps are translation, rotation and scaling, which can be applied to open and closed paths. Regardless of the curve shape, size and the relative distance, they demonstrated that following this approach the error estimation is enhanced. Turning functions were developed by Nadal et al. [18] to compare the shapes of two curves. Despite all these advances and improvements, the main weakness of LS optimization is implicit in the name itself. In fact, local search methods only explore the space of the variables in the neighborhood of the initial guess. So different initial guesses produce different local minima and there is no guarantee that the solution achieved in each case is the best one. Furthermore, depending on the quality of the initial guess, these methods are susceptible to failure or to carrying out a suboptimal search in the space of the variables.

In recent times, Evolutionary Algorithms have appeared as a valuable alternative to LS methods in optimization, making it possible to overcome the aforementioned obstacles. The capability of exploring the solution space over a wide area regardless of an initial guess is one of the main strengths of this approach. Furthermore, it is a derivative-free optimization method, which means that derivative information is unnecessary to find the optimal solution. This fact makes this approach more generalizable and enables problems to be tackled where the derivative of the goal function is unavailable. Furthermore, evolutionary optimization is a stochastic approach that introduces randomness into the search process. Randomness makes the algorithm less sensitive to modeling errors and allows the method to escape from optimum local solutions, enabling a search for the global solution.

EA includes different approaches such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Ant Colony (AC), Differential Evolution (DE), etc. GAs are based on natural biological evolution and are widely used in dimensional synthesis [19]. The first works using this approach appeared in the nineties and they were applied to four-bar linkages in path generation [20]. Multicriteria optimization is used in the works presented by Kunjur and Krishnamurthy [21], where the requirements of the linkages are extended to more realistic engineering problems. Cabrera et al. [22] considered the positions of the input link as design variables and introduced constraints in the algorithm to enable a progressive sequence of motion. In [23], the Ant-gradient algorithm is applied to the synthesis of a four-bar mechanism in path generation. In order to avoid some typical problems in mechanical design, PSO was used by Sardashti et al. [24]. However, DE has probably been the most widely used EA in recent years [25]. The work presented in [26] shows the use of DE combined with the geometric centroid of precision positions. It used penalty functions to introduce constraint violation and relative accuracy. Multiobjective optimization is dealt with by Cabrera et al. in [27] where additional constraints are included in the DE algorithm (e.g. velocity, acceleration or weight). DE is used with variable-controlled deviation to obtain the dimensions of a four-bar linkage in [28]. The three classical problems are tackled by Peñuñuri et al. [29] using DE, including the synthesis of the Ackerman mechanism. In [30], the Circular Proximity Function is implemented in a four-bar linkage using DE. In an interesting approach, it takes advantage of the feature that moving joints trace a circular curve. One of the main problems that EAs present is the selection of the control parameters. This selection is not obvious and different parameters could provide different results with different accuracy and efficiency. To prevent premature convergence Ortiz et al. [31] presented an auto-adaptive algorithm for the selection of control parameters and a new mutation operator to solve the problem of stagnation.

From the review of the literature, it was observed that most researchers in dimensional synthesis have abandoned their research work on local optimization and migrated to evolutionary algorithms. However, the advantages of LS versus EA should not be forgotten. In fact, EAs are efficient at exploring the entire search space; however, they take a relatively long time exploring the region to which the algorithm converges. To improve the lack of efficiency of EAs, new approaches have been developed using so-called Hybrid Algorithms [32,33]. By HAs we mean the combination of LS procedures with EAs. HAs take advantage of LS, which is good at finding local optima (local exploiter) and EA, which is good at exploring the search space (global explorer). Recently, Zhang et al. [34] presented a Hybrid Lagrange Interpolation Differential Evolution (HLIDE) algorithm for the study of path synthesis in four-bar linkages. Nevertheless, the application of LS to all individuals generated by EA not only leads to an increase in computational time but also a waste of time. In fact, many of the individuals generated by EA belong to the same local minima providing the same solution after LS. To overcome this problem, cluster-

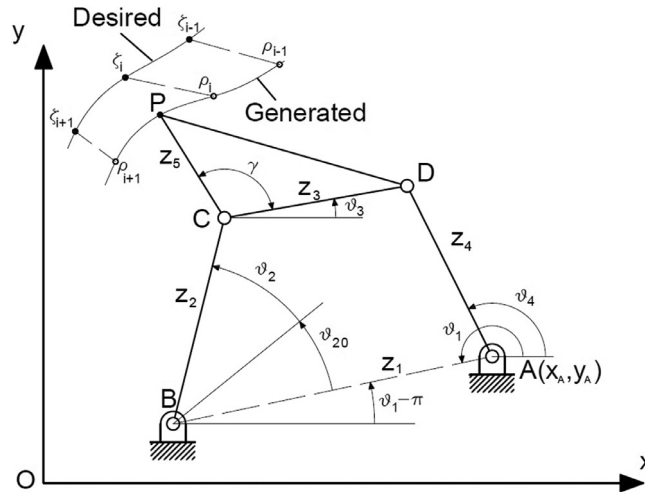


Fig. 1. Scheme of a four-bar linkage and coordinates used in the algorithm.

analysis uses Artificial Intelligent (AI) to create groups of similar individuals that may correspond to the same relevant region of the space. Recent literature [35–39] classifies clustering algorithms into three categories: hierarchical, partitional and overlapping. Hierarchical algorithms provide a tree structure representing the nested grouping of elements. Overlapping methods use fuzzy techniques to assign each element to the respective cluster. Partitional methods assign the groups based on certain criteria known as fitness measures. The K-means algorithm is the most fundamental of the partitional methods [40] and is popular due to its simplicity and lower computational cost.

This paper proposes a hybrid approach to solve the synthesis problem by incorporating a LS method into a DE approach, investigating interconnection by means of clustering and elite-based strategies. Elite-based strategies are intuitive approaches because they select the best individuals to perform the local search. However, efficiency could be reduced by selecting individuals that lead to the same solution in LS. On the other hand, methods based on cluster-analysis try improve efficiency and constitute a promising alternative in this kind of problems [33]. Although HAs have been widely applied in other scientific and engineering fields, their use in dimensional synthesis has not been sufficiently explored [29,32,34]. Therefore, the main objective of this work is to explore the use of a HA application to enhance the accuracy of synthesis methods considering their influence on efficiency. To this end, two hybrid algorithms have been developed and tested, one of them based on elite and another one based on cluster-analysis. In the paper both of them are compared with a pure DE algorithm to highlight their features and select the best one.

The new method should be robust and efficient, and for this reason, two new strategies to evaluate the fitness are incorporated in this study. A reordering step has been incorporated to those presented in a previous work [15]. Likewise, a new technique based on centroids and moments of inertia is presented. Improvements have been added by using an integrated optimization algorithm. This algorithm provides a better comparison of the desired and generated functions. This enables the convergence in a wide range of situations in planar mechanisms to be studied.

2. Goal function and optimization

Fig. 1 depicts a four-bar mechanism and all the geometrical parameters (design variables and coordinates) involved in the optimization problem. This scheme is used to illustrate the mathematical development presented in the following paragraphs, however, it should only be considered as an example to aid understanding of the concepts, since such a formulation can be applied to any kind of mechanism and synthesis problem.

2.1. Kinematic model and optimization

The problem presented in Fig. 1 is a path generation synthesis one, where p denotes the position of the coupler point with x_p and y_p coordinates defined in the Oxy system. The variation of these coordinates over time describes the generated path. The path is discretized at n_p precision points, which are generically denoted as r_i . The desired path is also discretized at the same number of prescribed points and is denoted as ζ_i . The error is part of the goal function and quantifies the difference between the specified path points on the desired curve and the generated ones. Thus, the error can be quantified at each precision point as,

$$\epsilon_i = \left\{ \begin{matrix} x_{di} - x_{pi} \\ y_{di} - y_{pi} \end{matrix} \right\}; i = 1, 2, \dots, n_p \quad (1)$$

where subscript i indicates the number of the precision point and subscripts d and p stand for desired and generated, respectively. The Synthesis Error (SE) function that has been widely used for path generation in the design of mechanisms is the cumulative square of the Euclidean distance. This error is given by,

$$SE = \frac{1}{N} \sum_{i=1}^{n_p} \mu_i \mathbf{e}_i^T \mathbf{e}_i = \frac{1}{N} \boldsymbol{\varphi}^T \boldsymbol{\mu} \boldsymbol{\varphi} \quad (2)$$

where

$$\boldsymbol{\varphi}^T = [\mathbf{e}_1^T \quad \mathbf{e}_2^T \quad \dots \quad \mathbf{e}_{n_p}^T] \quad (3)$$

and matrix $\boldsymbol{\mu}$ is a diagonal matrix including the weighting parameters for each precision point, i.e. μ_i . Normally, these weighting parameters are normalized so that they sum up to 1.

On the other hand, describing the linkage motion as well as the specified motion trajectories involves the formulation of kinematic constraint equations. These constraints are formulated using the Oxy reference frame shown in Fig. 1, obtaining the first two equations from the closed loop of the four-bar linkage (i.e. links z_1 , z_2 , z_3 and z_4). The other two equations provide the position of the coupler point P projecting the dimensions of the links z_1 , z_2 and z_5 onto the reference frame. That is,

$$\mathbf{C}_i = \begin{Bmatrix} z_1 \cos(\vartheta_1) - z_2 \cos(\vartheta_1 + \vartheta_{20} + \vartheta_2) + z_3 \cos(\vartheta_3) - z_4 \cos(\vartheta_4) \\ z_1 \sin(\vartheta_1) - z_2 \sin(\vartheta_1 + \vartheta_{20} + \vartheta_2) + z_3 \sin(\vartheta_3) - z_4 \sin(\vartheta_4) \\ x_p - z_5 \cos(\vartheta_3 + \gamma) + z_2 \cos(\vartheta_1 + \vartheta_{20} + \vartheta_2) - z_1 \cos(\vartheta_1) - x_A \\ y_p - z_5 \sin(\vartheta_3 + \gamma) + z_2 \sin(\vartheta_1 + \vartheta_{20} + \vartheta_2) - z_1 \sin(\vartheta_1) - y_A \end{Bmatrix}_i = \mathbf{0} \quad (4)$$

Geometrical constraints depend on the so-called vector of dependent coordinates, which must be determined for each position of the linkage,

$$\mathbf{q}_i^T = [\vartheta_{3i} \quad \vartheta_{4i} \quad x_{pi} \quad y_{pi}] \quad (5)$$

and the vector of design variables describing the position of the input link and geometrical dimensions of the linkage,

$$\mathbf{w}^T = [x_A \quad y_A \quad \vartheta_1 \quad \vartheta_{20} \quad \boldsymbol{\vartheta}_2^T \quad \mathbf{z}^T] \quad (6)$$

where the subvector $\mathbf{z}^T$ is formed by the dimensions of the links. That is,

$$\mathbf{z}^T = [z_1 \quad z_2 \quad z_3 \quad z_4 \quad z_5 \quad \gamma] \quad (7)$$

and the subvector,

$$\boldsymbol{\vartheta}_2^T = [\vartheta_{21} \quad \vartheta_{22} \quad \dots \quad \vartheta_{2n_p}] \quad (8)$$

contains the successive positions of the input link. Thus, the required design variables in this problem are the global position of the linkage $A(x_A, y_A)$, its orientation, ϑ_1 , the initial position of input link, ϑ_{20} , the sequence of angles of the same link, $\boldsymbol{\vartheta}_2$, and the dimensions of the links (i.e. z_j terms). The objective of the synthesis problem is to find the vector of design variables, $\mathbf{w}$, for which the coupler point P tracks the desired path with a minimum error. This error is measured by Eq. (2) which represents the basic form of the objective function. Consequently, the function must be minimized to find the dimensions of the four-bar linkage (i.e. vector $\mathbf{w}$) subject to a set of constraints which represent the geometrical requirements in Eq. (4). That is,

$$\begin{aligned} &\text{Minimize} && SE = \frac{1}{N} \boldsymbol{\varphi}(\mathbf{w}, \mathbf{q})^T \boldsymbol{\mu} \boldsymbol{\varphi}(\mathbf{w}, \mathbf{q}) \\ &\text{subject to} && \mathbf{C}_i(\mathbf{w}, \mathbf{q}_i) = 0; \quad i = 1, 2, \dots, n_p \\ &\text{and} && 0 < \vartheta_{21} < \vartheta_{22} < \dots < \vartheta_{2n_p} \end{aligned} \quad (9)$$

In Eq. (9), the dependence of the design variables and dependent coordinates has been highlighted. It is important to point out that the vector of the dependent coordinates depends on the design variables (i.e. $\mathbf{q}(\mathbf{w})$), but, for the sake of simplicity, this dependence has been omitted in this equation. Other constraints could be added to the basic problem, including the Grashof condition or the limits for the variation of the design variables.

2.2. Local search approach

The local search is an iterative process that uses gradient information to discover points that reduce the value of the goal function in the vicinity of the current solution. If a better solution is found, the current solution is replaced by it. This process is repeated until a local minimum is located. In this work the so-called Generalized Reduced Gradient method is used [5], which has been presented in detail in other previous works [13,14]. The recursive formula is obtained by means of differentiating SE function with respect to the vector of the design variables and equating it to zero. Therefore,

$$\mathbf{G}(\mathbf{w}, \mathbf{q})^T \boldsymbol{\mu} \boldsymbol{\varphi}(\mathbf{w}, \mathbf{q}) = 0 \quad (10)$$

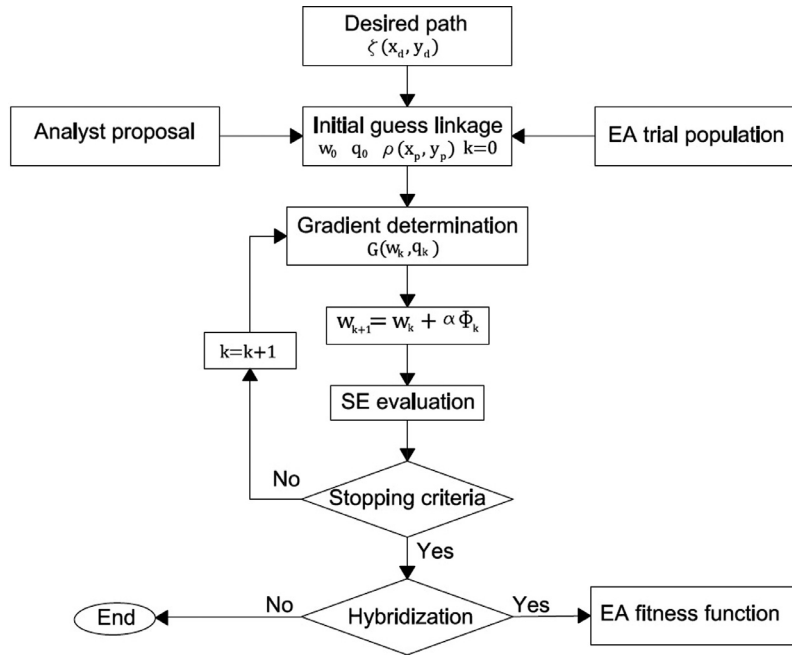


Fig. 2. Flowchart of the local search procedure.

where matrix $\mathbf{G}(\mathbf{w}, \mathbf{q})$ represents the gradient of the error. Now, using Taylor series expansion until the first derivative, Eq. (10) becomes,

$$\mathbf{G}(\mathbf{w}, \mathbf{q})^T \boldsymbol{\mu} \boldsymbol{\varphi}(\mathbf{w}, \mathbf{q}) + \mathbf{G}(\mathbf{w}, \mathbf{q})^T \boldsymbol{\mu} \mathbf{G}(\mathbf{w}, \mathbf{q}) \Delta \mathbf{w} = 0 \quad (11)$$

Adding subscripts and reordering this equation yields,

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha [\mathbf{G}(\mathbf{w}_k, \mathbf{q}_k)^T \boldsymbol{\mu} \mathbf{G}(\mathbf{w}_k, \mathbf{q}_k)]^{-1} \mathbf{G}(\mathbf{w}_k, \mathbf{q}_k)^T \boldsymbol{\mu} \boldsymbol{\varphi}_k = \mathbf{w}_k + \alpha \boldsymbol{\phi}_k \quad (12)$$

This equation represents the iterative process used to find the local minimum where subscript k stands for the iteration number, $\boldsymbol{\phi}_k$ is the direction of descent, and α is the step size. However, in its present form, this equation does not include constraints and the gradient matrix has to be obtained. The GRG algorithm solves these two problems at once. In fact, it differentiates the constraint in order to obtain the gradient. Thus, applying the chain rule to the equality constraints in Eq. (9) yields,

$$\frac{\partial \mathbf{C}}{\partial \mathbf{q}} \frac{d\mathbf{q}}{d\mathbf{w}} + \frac{\partial \mathbf{C}}{\partial \mathbf{w}} = \mathbf{0} \quad (13)$$

Reordering Eq. (13) enables the gradient to be obtained. That is,

$$\mathbf{G}(\mathbf{w}, \mathbf{q}) = \frac{d\mathbf{q}}{d\mathbf{w}} = - \left[\frac{\partial \mathbf{C}}{\partial \mathbf{q}} \right]^{-1} \frac{\partial \mathbf{C}}{\partial \mathbf{w}} \quad (14)$$

Eq. (13) provides the necessary information to use Eq. (12) iteratively until convergence.

The GRG algorithm also considers the inequality constraints included in Eq. (9). To this end, inequality constraints should be converted to equality constraints by means of using so-called slack variables [5]. Once the inequality constraints have been transformed to equality constraints, they are added to $\mathbf{C}_i$ and the gradient is determined in the manner shown above for equality constraints.

Fig. 2 shows the flowchart of the local search algorithm following the aforementioned steps. As will be explained later, this algorithm can work alone or together with an EA in so-called hybrid algorithms. When the algorithm works alone, the analyst must propose an initial guess to obtain the closest solution. However, in the case of hybridization, the evolutionary algorithm provides the initial guess.

2.3. Differential evolution algorithm

Under the EA denomination, several algorithms mimic natural phenomena to obtain an optimal solution. In this work, Differential Evolution has been selected as the most appropriate one because its features make it reliable, effective and robust in dimensional synthesis [29,31,32]. Although DE is quite similar to GA, it uses an adaptive scheme to ensure that the

alteration of the variables is automatically scaled, correcting magnitudes. This section describes the basics of the differential evolution algorithm implemented for solving the synthesis problem. The evolutionary strategy involves the evolution of a population with size s_{max} (i.e. the maximum number of individuals in each generation). This population is defined as follows,

$$\mathbf{w}_s^g = \mathbf{w}_{s,m}^g; \quad m = 1, 2, \dots, m_{max} \quad (15)$$

Superscript g stands for the generation number, subscript s is the index for one individual in its generation and m stands for the dimension of vector $\mathbf{w}$ (i.e. number of variables defining one individual). Thus, one individual is defined by the vector of design variables presented in Eq. (6) adopting the indexes shown in Eq. (15). Therefore, the size of one individual is given by m_{max} which coincides with the size of vector $\mathbf{w}$ in Eq. (6). The size of the population, s_{max} , does not change during the optimization process. The maximum number of generations, g_{max} , is established as a limit for the convergence of the algorithm. In other words, when g_{max} is reached, the algorithm stops regardless of the value of the fitness function. The fitness function is in this case equivalent to the goal function SE defined for local search optimization, i.e. Eq. (2). It measures the quality of the individual given by Eq. (15). However, DE does not directly allow the formulation of the constraints presented in Eq. (9) and they must be adapted to the DE algorithm. The method used in this work is the well-known Static Penalty [41], which is one of the most common approaches in EAs. Thus, the penalty term is included in the goal function and determined from the amount of constraint violation. That is,

$$\Phi(\mathbf{w}_s^g) = SE(\mathbf{w}_s^g) + \sum_{c=1}^{nc} R_c \max[0, g_c(\mathbf{w}_s^g)] \quad (16)$$

where R_c is the penalty coefficient, nc is the number of constraints and g_c is the constraint itself. The penalty term penalizes the goal function [42]. Thus, if equality constraints are satisfied no violation occurs and the penalty term will be zero. The inequality constraints will be formulated in such a way that g_c is negative when they are satisfied. In this way the penalty term will be zero. Otherwise, it provides a positive value of the penalty term, proportional to the magnitude of the violation of the constraint.

The initial vector population is randomly chosen to cover the entire space of the design variables. In this work, a uniform probability distribution has been selected to provide a value of the variables within the dimensional limits established by the analyst. Therefore,

$$w_m = rand_m(w_m^U - w_m^L) + w_m^L; \quad m = 1, 2, \dots, m_{max} \quad (17)$$

where $rand_m$ is a random number between 0 and 1 with a uniform distribution, and subscripts U and L stand for the maximum and minimum limits of the design variables, respectively. Once the initial population has been generated, the evolutionary process begins, providing better individuals in each generation. This process is achieved by means of the so-called operators, which in this work are mutation, crossover and selection. One of the main differences between DE and GA is the mutation operator. Indeed, mutation is carried out using three individuals to obtain a mutant factor. One of them is called the base operator, which is perturbed by the difference between the other two. Thus, for each individual in each generation the mutant vector is obtained using the following equation,

$$\mathbf{v}_{s,m}^{g+1} = \mathbf{w}_{s1,m}^g + F(\mathbf{w}_{s2,m}^g - \mathbf{w}_{s3,m}^g); \quad s = 1, 2, \dots, s_{max} \quad (18)$$

The subscripts $s1, s2$ and $s3$ are randomly selected among the population and are different from each other. Parameter $Fv[0, 2]$ is a constant factor selected by the user, controlling the difference between vectors $s2$ and $s3$. Crossover is the next operator applied to the evolutionary process using the following scheme,

$$\mathbf{u}_{s,m}^{g+1} = \begin{cases} \mathbf{v}_{s,m}^{g+1} & \text{if } A(m) \leq Cr \\ \mathbf{w}_{s1,m}^g & \text{if } A(m) > Cr \end{cases}; \quad m = 1, 2, \dots, m_{max} \quad (19)$$

where $Crv[0, 1]$ is the crossover constant which must be established by the analyst, $A(m)$ is a random number. The next step is selection, which is the final operator to be applied in the DE algorithm. It decides whether an individual is preserved for the next generation, otherwise, a new one is selected using a greedy criterion. This criterion is given by,

$$\mathbf{w}_s^{g+1} = \begin{cases} \mathbf{u}_s^{g+1} & \text{if } \Phi(\mathbf{u}_s^{g+1}) < \Phi(\mathbf{w}_s^g) \\ \mathbf{w}_s^g & \text{if } \Phi(\mathbf{u}_s^{g+1}) \geq \Phi(\mathbf{w}_s^g) \end{cases} \quad (20)$$

Eq. (20) is repeated for each generation until the g_{max} value is achieved. The use of an error threshold is also possible as a convergence criterion, however, the former criterion is preferred in this work due to the uncertainty that the latter produces in this kind of algorithms. In summary, five parameters are necessary to apply the algorithm, i.e. population size, s_{max} , constant factor, F , crossover constant, Cr , constraint factor, R_c , and maximum number of generations, g_{max} .

3. Fitness evaluation

In the majority of works the comparison of the two paths is provided by the Euclidean distance, which could be a source of inaccuracies, especially when DE algorithms are used [18]. The main disadvantage of SE is that it calculates the distances

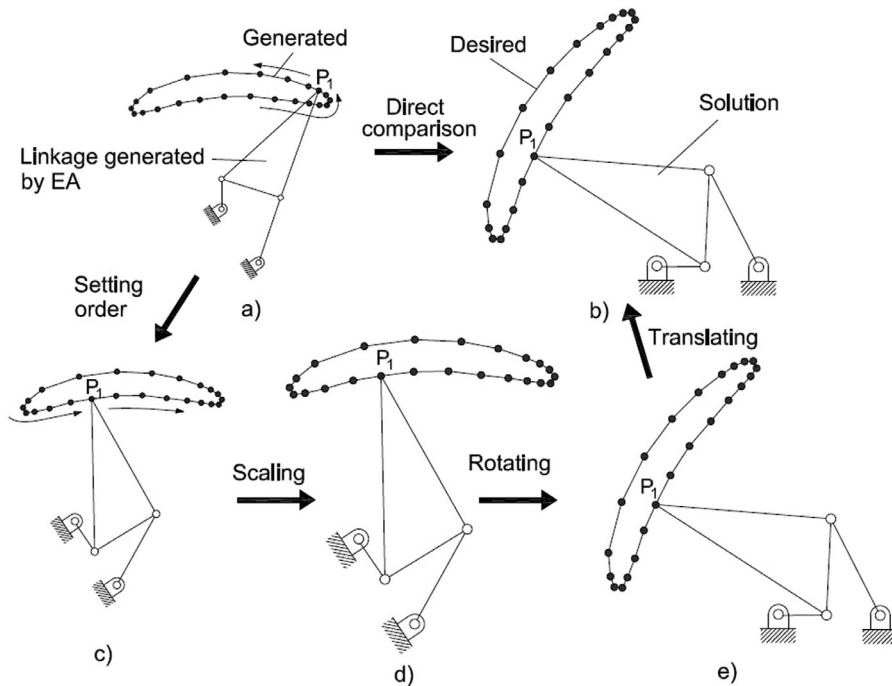


Fig. 3. Transformations involved in the comparison of paths.

point-by-point between the two curves and this magnitude depends on several factors such as the points' correlation, linkage location, relative orientation and scaling. In Sections 3.1 and 3.2, the two new approaches for reducing the errors are presented. However, previously to this description, clarification of some concepts is necessary, which is done in the next paragraphs.

Fig. 3a shows an example of a four-bar linkage provided by an EA within an iterative process. Fig. 3b shows the desired path and the exact solution, that is, the mechanism generating a path that perfectly fits the desired one. The direct application of the SE function to compare both curves (generated and desired) would produce a high value of the error. However, a closer analysis of both curves reveals that they are not so different. Indeed, if some basic transformations are applied to the linkage in Fig. 3a, the mechanism in Fig. 3b is obtained and the error is reduced to zero. These transformations are the following:

- Reordering reference numbers: the index number one on the desired curve (point P1 in Fig. 3b) is randomly selected by the analyst or the algorithm. The rest of the points are ordered in a correlative way in the direction in which the linkage is expected to move until the last prescribed point is reached (i.e. index np). The algorithm gives the index numbers of the precision points in the generated curves. This fact assumes that there is a mismatch between similar points in both curves due to the random numeration of discrete points. Reordering the numbers enables the differences between both curves to be reduced, as is shown in Fig. 3c where the point P1 has been selected as the most appropriate point to initiate the counting, reducing the error computed by Eq. (9).
- Scaling: the size of the path can be modified by scaling the size of the mechanism using a scaling factor. This transformation is quite simple and provides a generated path that is similar in size to the desired one, as is shown in Fig. 3d.
- Rotation: the rotation of the generated linkage also contributes to the reduction of the error provided by the SE function. Fig. 3e shows this transformation.
- Translation: the translation of the generated linkage allows one curve to be brought towards the other, reducing the estimation of the error due to the relative distance between them. In the example in Fig. 3, translation makes both curves coincident.

Then, in the example in Fig. 3, the final path perfectly fits the desired one once the four transformations have been applied and SE is equal to 0. Nevertheless, during the optimization process the EA produces linkages whose error is reduced after transformations but is not removed (i.e. $SE \neq 0$). However, SE is reduced in each generation. Before using Euclidean distance, two methods are proposed in this paper to approximate the generated path to the desired one. The first one is an approximate approach, which is based on the use of centroids and moments of inertia. The second one is based on local search optimization. They are described in the following subsections.

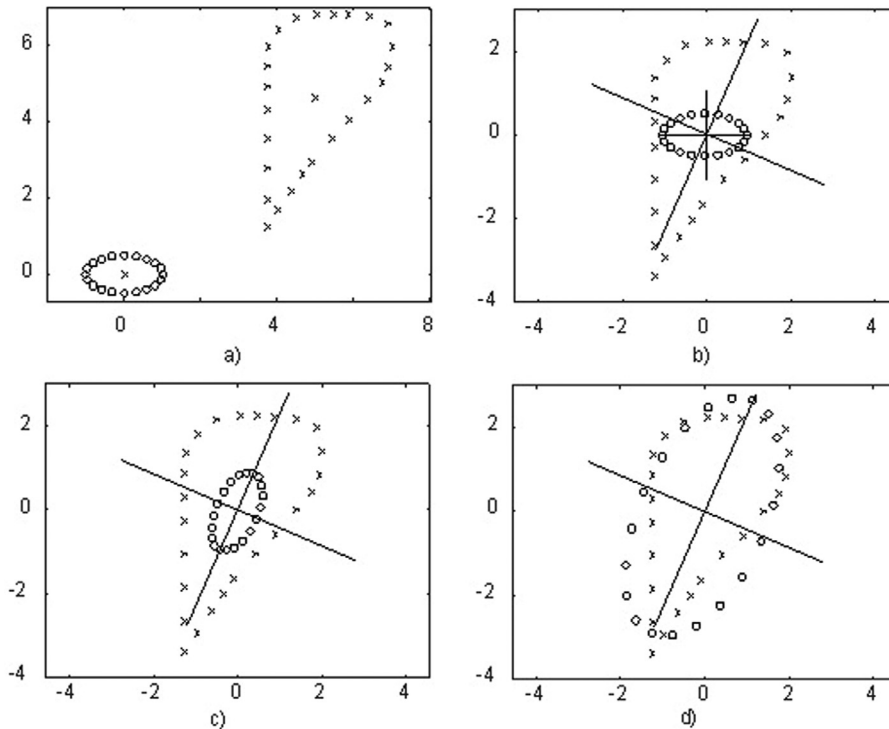


Fig. 4. Transformations based on centroids and inertia matrix: a) initial position, b) translation, c) rotation and d) scaling.

3.1. Fitness evaluation based on centroids and moments of inertia (FECMI)

An easy way to approximate the generated path to the desired one is to use the centroids and moments of inertia of both curves. This approach is a numerical treatment that bears little theoretical meaning. Thus, the accuracy of the results is not as good as using other approaches; however, from the efficiency viewpoint, the reduced computational time required makes this method attractive. The first step is to calculate the centroids of both desired and generated curves. This step is carried out considering that the weighting parameters are equivalent to the mass of each precision point. That is,

$$x_c = \frac{\sum_{i=1}^{n_p} \mu_i x_i}{\sum_{i=1}^{n_p} \mu_i}, \quad y_c = \frac{\sum_{i=1}^{n_p} \mu_i y_i}{\sum_{i=1}^{n_p} \mu_i} \quad (21)$$

where x_c and y_c are the Cartesian coordinates of the centroid. Then, the curves are translated taking the centroid as the reference point to make them coincident. Fig. 4a shows an example with two different curves and their respective centroids, and Fig. 4b shows both curves translated to a common point making their centroids coincident. The moments of inertia of both curves are used for the rotation of the generated path. To this end, the principal axes are calculated as if they were rigid bodies by using the inertia matrix of both curves. This requires the solution of a problem of eigenvectors and eigenvalues considering the precision points as particles. The problem to be solved for the desired path is given by,

$$(\mathbf{H}_d - \sigma_d \mathbf{I}) \mathbf{v}_d = 0 \quad (22)$$

where $\mathbf{H}_d$ is the inertia matrix, $\mathbf{I}$ is the identity matrix and σ_d and $\mathbf{v}_d$ are the eigenvalues and eigenvectors, respectively. The eigenvectors give the directions of the principal axes of inertia, which enable rotation of the generated path. In Fig. 4b the principal axes obtained from the eigenvectors are shown. In Fig. 4c one curve is rotated in order to make the principal axes of both curves coincident. In a similar way, scaling is obtained using polar moments of inertia,

$$\tau = \frac{I_{0d}}{I_{0g}} \quad (23)$$

where τ is the scaling factor, and I_{0d} and I_{0g} are the polar moments of inertia of desired and generated curves, respectively. Fig. 4d shows both curves after the scaling factor is applied to one of them. Finally, renumbering the points makes the two curves ready to be compared accurately by means of the Euclidean distance.

Fig. 5 shows the flowchart used for the DE algorithm. The process starts with the definition of the desired function, then the four parameters of the DE algorithm are defined and the initial population is generated. All individuals in each generation are processed through the FECMI algorithm to approximate both functions. In this way, fitness evaluation is

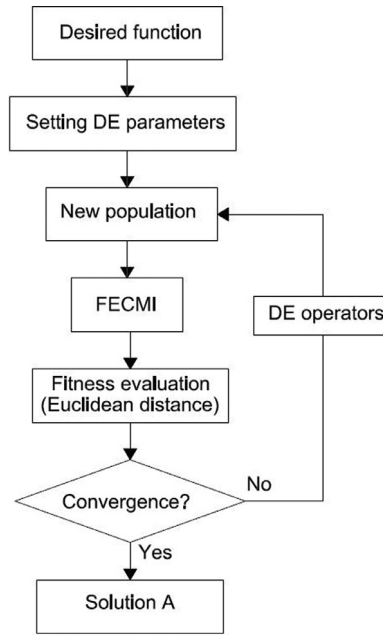


Fig. 5. Implementation of FECMI in an EA.

obtained by means of the Euclidean Distance under the best possible conditions. Solution A is a set of linkages ordered by the fitness from the best to the worst. Obviously, the mechanism with the lowest synthesis error is considered to be the solution.

3.2. CERED algorithm

The Comparative Error Reduction based on Euclidean Distance (CERED) approach is a well-founded algorithm, which includes the four transformations described in the last section. Because favorable conditions for local search are met, the CERED algorithm uses local search (GRG method) to approximate the generated path to the desired one. Starting from the linkage provided by the EA, the CERED algorithm searches for the closest solution. CERED transforms the vector of design variables into five vectors as follows,

$$\begin{aligned}
 \mathbf{w}_T^T &= [x_A \quad y_A] \\
 w_R &= \vartheta_1 \\
 w_P &= \vartheta_{20} \\
 \mathbf{w}_H^T &= [\vartheta_{21} \quad \vartheta_{22} \quad \dots \quad \vartheta_{2n_p}] \\
 w_C &= \tau
 \end{aligned} \tag{24}$$

Hereinafter vector $\mathbf{w}_T$ will be named translation vector because its modification only changes the position of the linkage with respect to the desired path. Vector w_R is the one governing the orientation of the linkage and for this reason it is denominated rotation vector. Vector w_P modifies the initial position of the input link (i.e. locates point P_1 in Fig. 3) and w_H establishes the sequence of positions of this link. The scaling factor is provided by w_C and gives the geometrical dimensions of the links scaled by a factor τ . The use of the scaling factor modifies the third and fourth constraints given in Eq. (4). That is,

$$\begin{cases} x_p - \tau [z_5 \cos(\vartheta_3 + \alpha) - z_2 \cos(\vartheta_1 + \vartheta_{20} + \vartheta_2) + z_1 \cos(\vartheta_1)] - x_A \\ y_p - \tau [z_5 \sin(\vartheta_3 + \alpha) - z_2 \sin(\vartheta_1 + \vartheta_{20} + \vartheta_2) + z_1 \sin(\vartheta_1)] - y_A \end{cases}_i = \mathbf{0} \tag{25}$$

The efficiency of the CERED algorithm in evaluating the SE function depends on the order in which the four steps are performed. To establish the most effective order, a trial-error study was carried out with the different alternatives of sequences. The result is presented in Fig. 6, where the connection with the EA is also shown. The CERED algorithm takes information from the trial population generated by the evolutionary algorithm. The linkages are first translated, then rotated, scaled, and finally, the initial position of the input link is determined. Each one of these steps involves a local search optimization using the GRG algorithm. The only difference in each step lies in the vector of design variables. The process is cumulative, which means that each step involves the previous one. Thus, translation, which is the first step, only involves its own variables. However, rotation involves its own variables and the variables in the previous step (i.e. translation). Reordering involves the

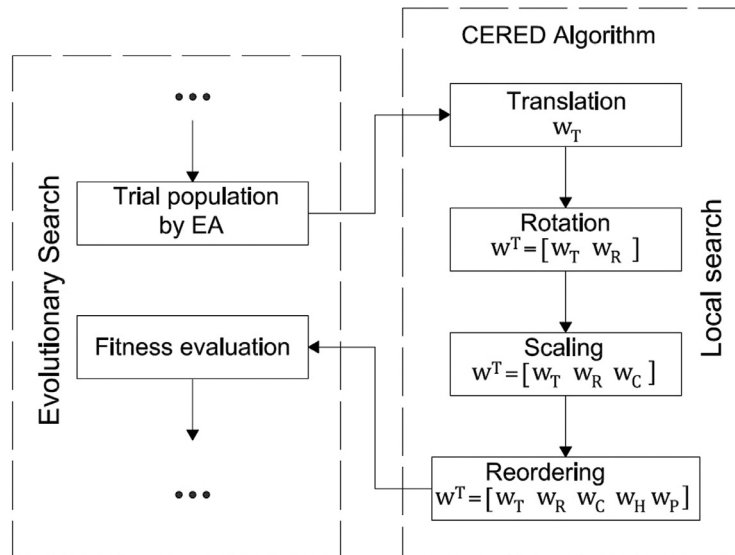


Fig. 6. Implementation of the CERED algorithm.

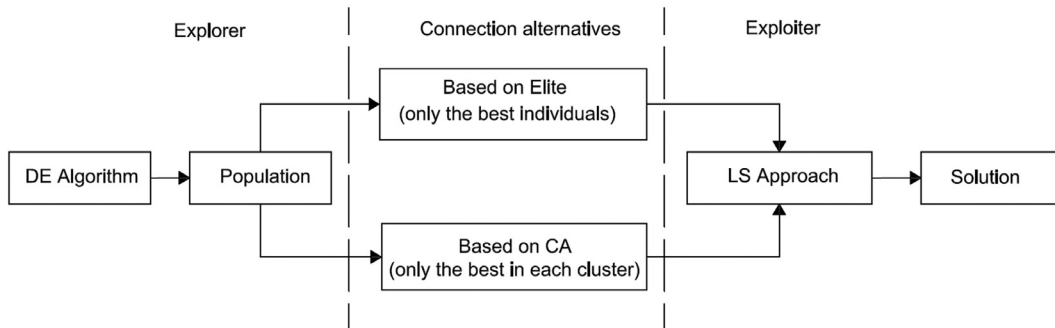


Fig. 7. Selection of individuals to be considered in local search. Two alternatives are shown, based on Elite and the Cluster Analysis.

sequence variables and the variables from translation and rotation. Finally, scaling implies all the design variables involved in the CERED algorithm. In any case, a small number of variables are involved in the local search making CERED an efficient algorithm.

4. Hybridization of LS and DE

Many works have shown that DE performs well for global search [35,37] because it is capable of quickly finding and exploring promising regions of the search space. However, it takes a relatively long time converging to a local minimum. On the other hand, the local search approach is good at finding local optima but it fails to explore the global search space. Thus, it is easy to deduce that a hybrid algorithm, which is based on the combination of both optimization procedures, enhances their performance. However, in order to obtain an efficient HA, it is necessary to reduce the time spent by local search. It is important to point out that it is not efficient to carry out a local optimization for every individual generated by DE. Furthermore, the application of the local search to every individual will provide many similar solutions, which implies a considerable waste of time. For this reason, one crucial step in defining a hybrid algorithm is to determine which individuals should take part in local optimization. Only the most promising ones should be used. One of the main goals pursued in selecting only a set of individuals is to reduce the computational cost of applying local optimization. It is important to highlight that efficiency is one of the weaknesses of EA and it should be controlled to prevent excessive time being used to achieve convergence.

This section presents the implementation of LS in a DE algorithm to enhance the exploitation ability of the evolutionary approach and to cooperate producing the desirable synergetic effect in the optimization process. Hybridization is applied with two aims. The first is to evaluate the fitness by using the CERED algorithm. The second is to obtain the best optimized mechanism. To this end, two approaches are applied to choose which individuals are selected for local search. These two approaches are shown in the scheme in Fig. 7. The first approach is based on elitism and it applies local optimization only

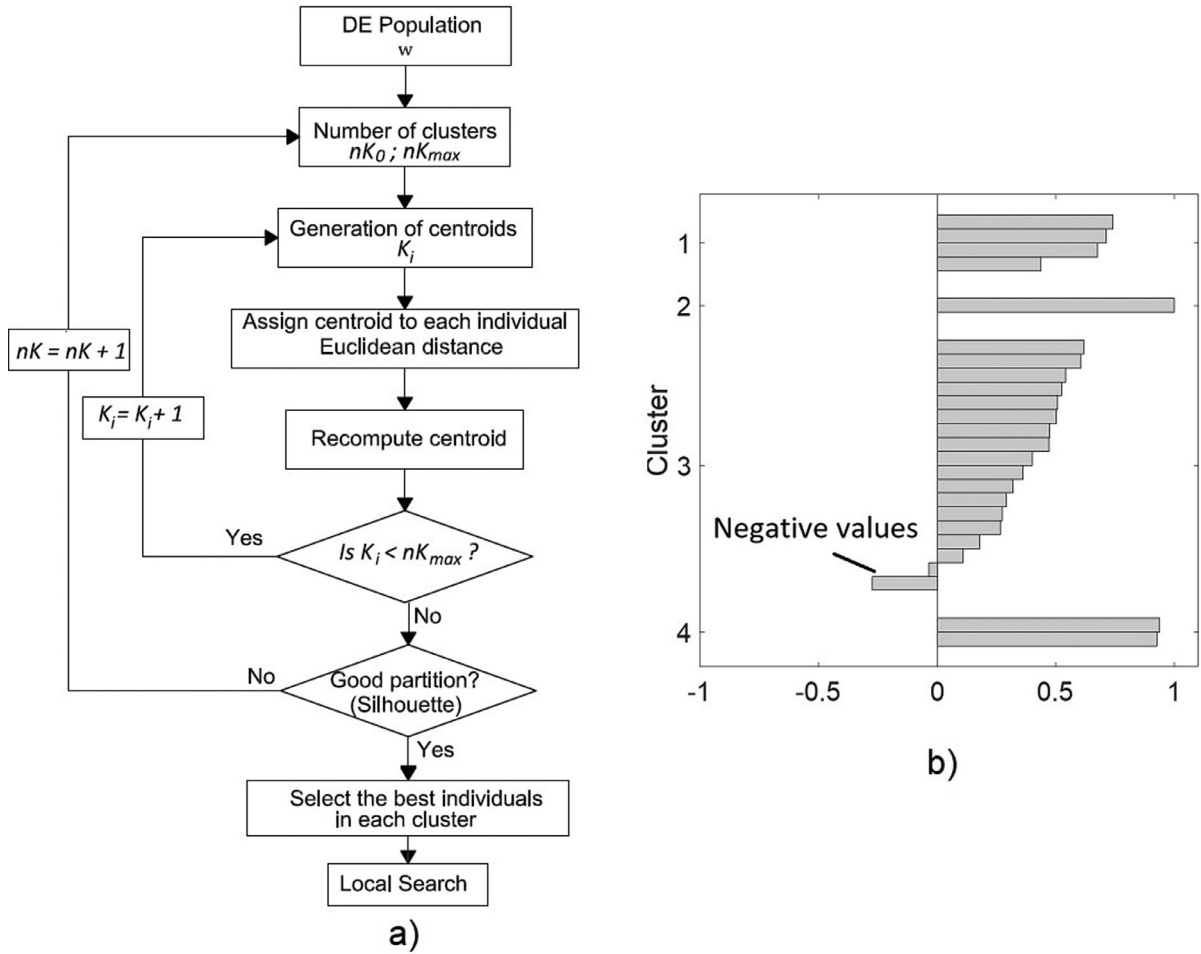


Fig. 8. Cluster analysis: a) algorithm used and b) example of silhouette.

to the best individuals selected as Elite in the DE algorithm. In the second approach a cluster analysis is used for grouping individuals with similar features. This approach is presented in the following subsection.

4.1. Data mining and cluster analysis

Cluster analysis is part of data mining science and applies a clustering algorithm to the population in order to find patterns among the individuals. Thus, a cluster is a group of individuals that belongs to the same class with similar features. After the analysis several groups (or clusters) of individuals are provided; those individuals belonging to different clusters are dissimilar. The measure of the similarity is a crucial factor allowing an individual to be assigned to the corresponding cluster. This measure can be formulated using either deterministic or probabilistic distance. Fig. 8a shows the scheme of partitioning in the K-means clustering algorithm, which is used in this work to obtain the clusters. It begins with the selection of a number of clusters (i.e. nK). Then, each cluster is associated with an initial guess centroid denoted as K_i . Each individual is then assigned to the closest initial guess centroid using Euclidean distance. Thus, a collection of individuals assigned to the same centroid form a cluster. However, the initial guess centroid is not the actual centroid (i.e. it is an initial guess) which has to be determined for each cluster. The assignment and centroid update steps are repeated in an iterative process until centroids remain the same, i.e. convergence is achieved. The silhouette (see Fig. 8b) of the clustering is a graphical method for the interpretation and validation of consistency within the clusters. The silhouette is a measure of the cohesion of the individual in its own group, and the difference with respect to other individuals in other groups (separation). The silhouette has the following form,

$$S(i) = \frac{D_{wi} - d_{wi}}{\max\{D_{wi}, d_{wi}\}} \quad (26)$$

where d_{wi} is the average distance of individual i to all other individuals in the same group, and D_{wi} is the average distance of individual i to all individuals in any other cluster. Thus, d_{wi} is a measure of how similar individual i is to its own group,

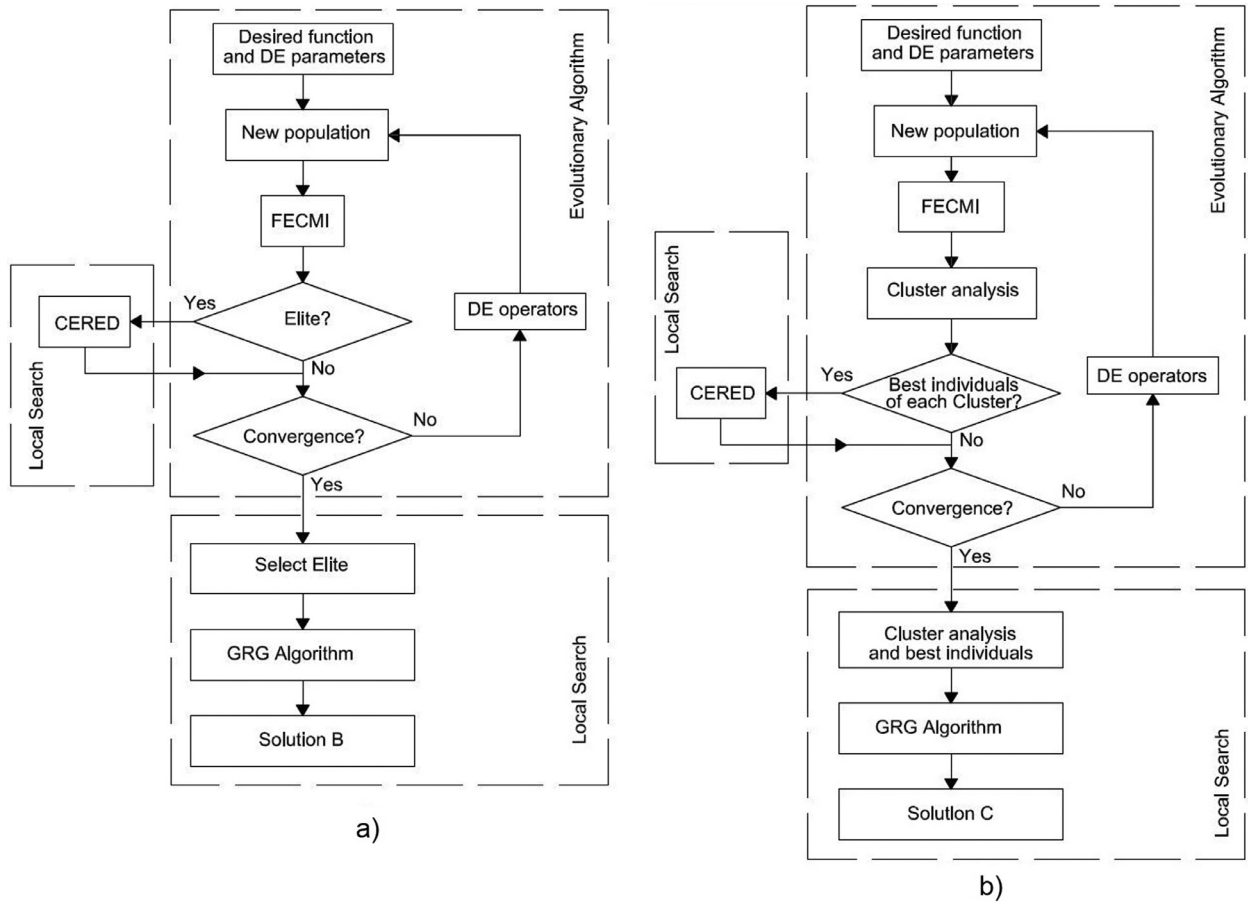


Fig. 9. Different strategies used in the research work a) DEE algorithm based on elite selection and b) DEC algorithm based on cluster analysis.

so it should be a small value. On the other hand, D_{wi} is a measure of how dissimilar individual i is to individuals in other groups, so this should be a large number to be considered well assigned. Thus, it is clear that the limits of the function can be established as,

$$-1 \leq S(i) \leq +1 \quad (27)$$

Negative values of the function imply that the individual is wrongly assigned. In contrast, values close to 1 mean that the individual is well matched. Values close to 0 mean that the individual is on the border. Fig. 8b shows an example of silhouette obtained during the optimization process where the negative values indicate that they are wrongly assigned. The main disadvantage of the method is that the analyst must provide the number of clusters. To solve the problem, it is recommended to repeat the process several times using different numbers of clusters (from nK_0 to nK_{max} , see Fig. 8a) until the values of the silhouette of the partition are as expected in a good assignment.

4.2. Models for hybridization

As has been shown in previous paragraphs, hybridization can be used in EA both to enhance the evaluation of the fitness function and to improve the exploitation of local minima. In this work all possible combinations are studied and compared with respect to the pure DE algorithm. To this end, three algorithms were developed and tested. They are the following:

- DE: This algorithm does not use local search at all, it is considered a pure DE algorithm that uses the FECMI approach to evaluate the fitness function. This algorithm is used as a reference in this work (see Fig 5).
- DEE: This algorithm is a DE approach that uses CERED only in the best individuals, i.e. the individuals selected as elite (E). For the rest of the individuals it uses FECMI (see Fig. 9a). To enhance the solution provided by DE, the algorithm applies LS only to the elite.
- DEC: This algorithm is a DE approach that uses CERED to evaluate the fitness function and enhances the solution provided by EA in those individuals selected by cluster analysis (C). The rest of the individuals are evaluated using FECMI (see Fig. 9b).

The results provided by this comparison enable the selection of the best algorithm considering its features in terms of accuracy, efficiency and robustness. Fig. 9a shows the flowchart for the DEE algorithm where the hybridization of AE and LS is highlighted. As can be observed, the fitness evaluation is based on Elite so that only the best individuals are submitted to the CERED algorithm, providing an enhancement of the accuracy while avoiding an important increment in time. At the convergence of the evolutionary algorithm, a family of linkages is provided, ordered by fitness values. Only the Elite of this family are selected to be optimized with the LS algorithm, providing the solution denominated B.

Fig. 9b shows the scheme of the DEC algorithm. In this case, K-mean analysis is carried out, grouping the generated individuals in clusters. Individuals in the same cluster are assumed to produce similar results after local search. Thus, only the best individual of each cluster is optimized using the CERED or GRG algorithm.

5. Synthesis examples

In this section the efficiency, accuracy and robustness of the three algorithms are studied and compared in three case studies. To this end, two different planar linkages and three different synthesis problems are considered. The operator's parameters of the DE remained constant in all cases. The probability of crossover was $Cr = 0.8$, $F = 0.7$, elite was selected from the 1% of the best population. In these problems both the size of the population and the number of generations were modified. The population sizes were established as 10, 25, 50, 100, 200 and 500. The maximum number of generations was 10, 20 and 30, which were also used as stopping criteria. All precision points were equally weighted. In order to take into account the randomness of the evolutionary algorithms, each case has been run 10 times and the results were statistically compared. All examples were run in Matlab® in a computer equipped with an Intel® Core™ i7-8700 processor running at 3.2 GHz.

5.1. Case study 1: path generation with a four-bar linkage and an open desired path

In this example, an open L-shaped curve is proposed as the desired path. Table 1 shows the Cartesian coordinates of the 11 precision points used in its definition. The mechanism proposed is a four-bar linkage whose kinematic scheme is shown in Fig. 1. The goal is to make the coupler point pass through the 11 prescribed points. The reason why this path has been selected is the special difficulty that a four-bar linkage has to generate it. The path generation problem is tackled with the three algorithms previously presented in this paper. The design variables are shown in Table 2 together with the best results obtained in the three cases. Fig. 10 shows the best mechanisms describing the paths. These examples show the results obtained under the best conditions for the algorithm, i.e. $g_{\max} = 30$ and $s_{\max} = 500$. Regardless of the number of generations, the DEC algorithm provides the best solutions. The accuracy of two of them is quite similar and therefore they are shown in Fig. 10a and b. Table 2 shows the corresponding SE values which are 1.8702 mm² and 1.8398 mm², respectively. Although they are similar values, the linkages are quite different and for this reason they are depicted in these two figures. Fig. 10c shows the best result obtained for the DEE algorithm providing a SE value of 2.0818 mm². Finally, Fig. 10d shows the best result obtained with the DE algorithm with a SE value of 2.0914 mm². These results show that DEC not only provides the best accuracy, but also gives different dimensions or alternatives with high quality. On the other hand, the qualitative differences between the results offered by DEE and DE are not important. Indeed, in a closer look at the figures, both linkages seem very similar. DEE achieves more accuracy because it has a better capability for exploitation.

The best results obtained for different sizes of population and different generations are shown in Fig. 11. This figure shows two kinds of results, providing valuable information about the accuracy and efficiency of the three approaches. Fig. 11a, c and e show the accuracy in terms of the SE value for $g_{\max}$ values of 10, 20 and 30, respectively. Fig. 11a and c show that when the number of generations and size of the population are low (i.e. 10 or 20 generations and 10 individuals), DEE provides the best solution. However, when the numbers grow the behavior change and the DEC algorithm offers

Table 1

Precision points used in the desired L-shaped path (units in mm).

Nº	1	2	3	4	5	6	7	8	9	10	11
x_d	0.0	0.0	0.0	0.0	0.0	0.0	3.0	6.0	9.0	12.0	15.0
y_d	15.0	12.0	9.0	6.0	3.0	0.0	0.0	0.0	0.0	0.0	0.0

Table 2

SE and design variables for the best results obtained with the three algorithms.

Algorithm	SE (mm ²)	Variables									
		x_A (mm)	y_A (mm)	ϑ_1 (deg)	ϑ_{20} (deg)	z_1 (mm)	z_2 (mm)	z_3 (mm)	z_4 (mm)	z_5 (mm)	γ (deg)
DEC	1.8702	0.1	29.7	56.34	−178.4	80.2	12.9	96.4	29.1	19.2	56.3
	1.8398	21.2	5.6	−116.2	−156.8	31.5	18.2	24.6	25.2	16.9	74.1
DEE	2.0818	11.8	15.1	35.5	165.4	19.9	14.0	90.5	92.1	3.8	−53.6
DE	2.0914	14.8	10.6	50.31	149.1	18.2	13.2	54.9	54.2	3.6	−125.5

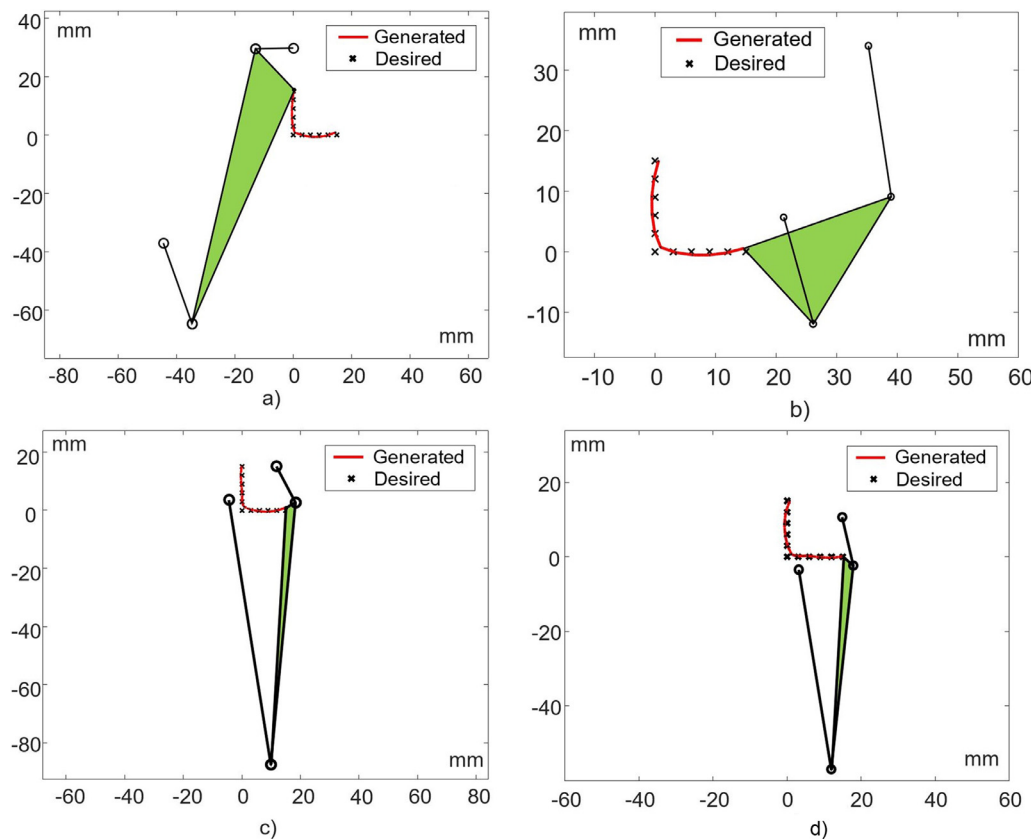


Fig. 10. Desired and generated paths and the corresponding mechanisms obtained at the convergence: a) best solution obtained with DEC, b) second best solution obtained with DEC, c) DEE best solution and d) best solution obtained with DE.

$g_{\max}$ Algorithm		Size of the population, $s_{\max}$					
		10	25	50	100	200	500
10	DE	0	1	0	1	0	3
	DEE	0	1	1	1	3	4
	DEC	0	1	2	3	4	5
20	DE	0	0	0	2	2	4
	DEE	1	0	1	3	3	5
	DEC	0	1	3	3	6	7
30	DE	0	1	2	3	2	4
	DEE	1	0	2	4	2	4
	DEC	2	2	3	4	4	8

the best solution. In many cases, there are no significant differences between DEE and DEC, but dissimilarities become important when the number of generations and the population increase in size. These differences are quite significant when the values of $g_{\max}$ are 20 and 30 with a population of 500 individuals. In this last case, the best general results are obtained.

In the same figure, the features of the efficiency of each algorithm are shown. As can be observed in Fig. 11b, d and f, the DE algorithm obtains the best CPU time in all cases. The reason for this is obvious; DE is a pure evolutionary algorithm that does not consume time making use of local search. On the other hand, the comparison of the DEE and DEC algorithms provides valuable information that is not obvious. Indeed, when the size of the population is low (i.e. from 10 to 100 individuals), the efficiency of DEE algorithm is greater, requiring less time to achieve convergence than DEC. Nevertheless, when the size of the population is larger than 100 individuals this trend changes and the efficiency of the DEC algorithm becomes more and more significant.

A measure of the robustness of the three algorithms is presented in Table 3. In this case, the trial is considered successful if the accuracy obtained is greater than or equal to 80% of the maximum accuracy (i.e. $SE = 1.8398 \text{ mm}^2$). The numbers in

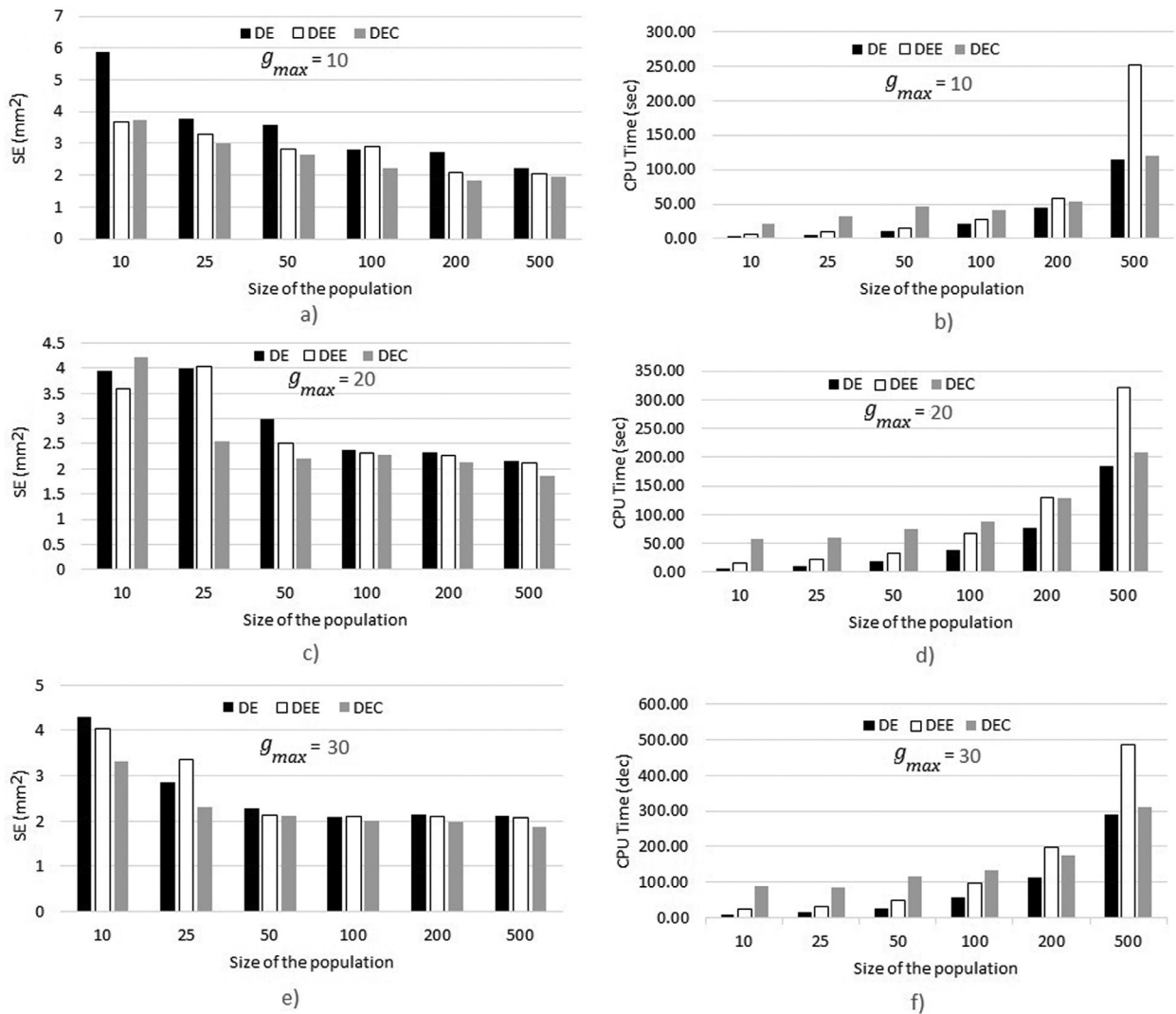


Fig. 11. Accuracy and CPU time of the results obtained in the synthesis of the L-shaped path.

Table 4

Precision points used in the desired closed path (units in mm).

Nº	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
x_d	14	13.9	13.5	12.8	11.8	10.8	9.8	9.3	8.7	8	7.5	7.5	7.5	7.5	7.5	7.5	7.5	7.6	8.1	9	10.1	11	11.7	12.8	13.8
y_d	11.9	10.9	10	9.2	8	7.1	5.8	5.2	4.4	3.3	2.4	3.9	5.5	7.1	8.6	9.8	10.9	11.9	12.8	13.5	13.7	13.6	13.6	13.6	13.1

the table represent the number of times that this accuracy has been achieved in 10 trials. Thus, the value zero in the cells means that none of the ten solutions in the trials achieved this value of accuracy. The best situation is obtained with DEC when $g_{\max} = 30$, $s_{\max} = 500$, where eight of the ten trials achieved this accuracy. In any case, the DEC algorithm provides the greatest robustness in almost all cases. Only when working with small populations is this not fulfilled.

5.2. Case study 2: path generation with a four-bar linkage and a closed desired path

In the second case, a closed path with 25 prescribed points is considered. The Cartesian coordinates of the points defining the desired curve are given in Table 4. The problem considers that the coupler point of a four-bar linkage must pass through the points. In this case, the Grashof condition is included in the formulation of the optimization problem.

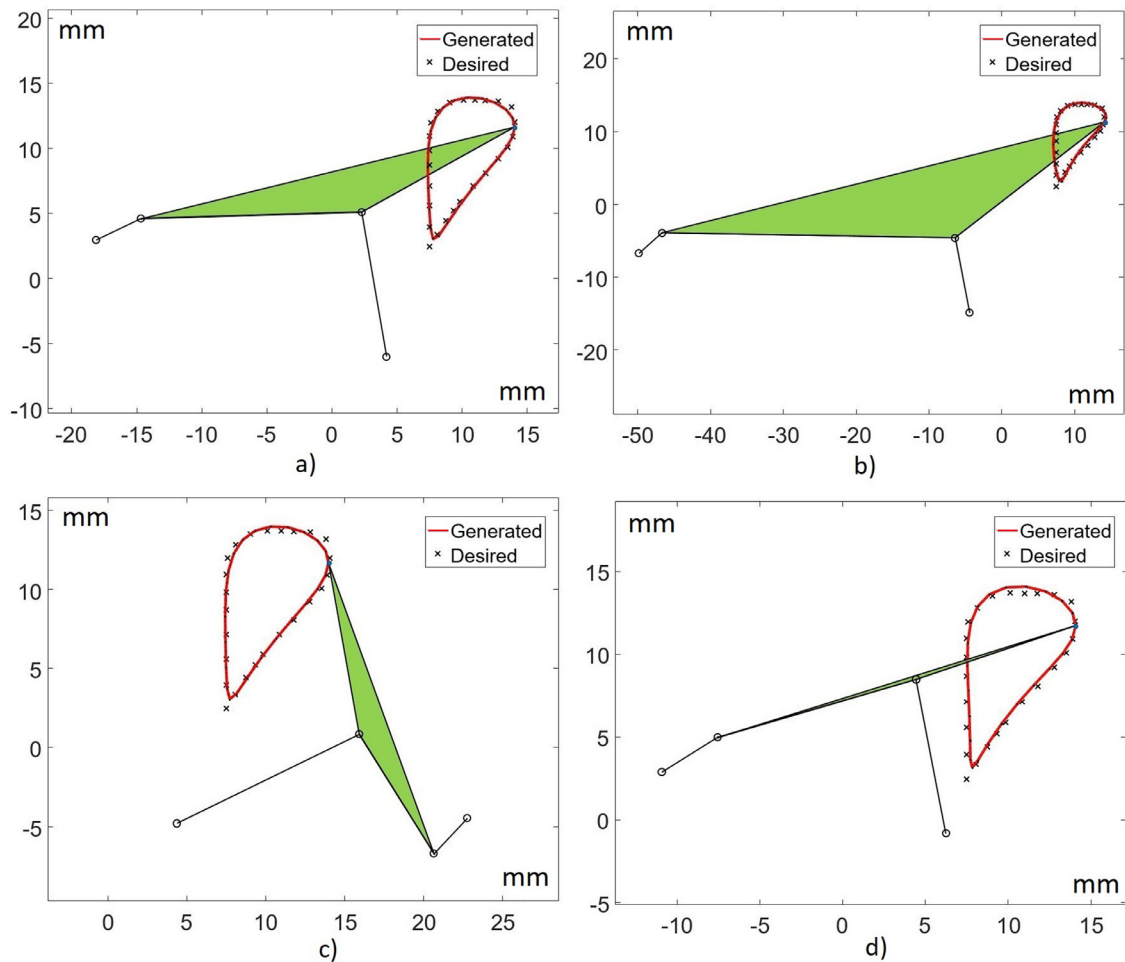


Fig. 12. Desired and generated paths and the corresponding mechanisms obtained at the convergence: a) best solution obtained with DEC, b) second best solution obtained with DEC, c) DEE best solution and d) best solution obtained with DE.

Table 5

Best results obtained with the three algorithms developed.

Algorithm	SE (mm ²)	Variables									
		x_A (mm)	y_A (mm)	ϑ_1 (deg)	ϑ_{20} (deg)	z_1 (mm)	z_2 (mm)	z_3 (mm)	z_4 (mm)	z_5 (mm)	γ (deg)
DEC	1.1039	22.1	-2.9	8.6	-123.1	18.8	2.9	7.9	14.0	18.0	-15.3
	1.1362	-18.1	2.9	158.1	25.5	24.0	3.8	16.9	11.2	29.5	12.0
DEE	1.5335	-10.9	2.8	167.8	31.9	17.5	3.9	12.4	9.4	22.6	1.0
DE	1.7308	-49.7	-6.7	169.8	41.8	46.1	4.2	40.2	10.4	62.6	15.0

The information about the four best results provided by the three algorithms is shown in Fig. 12 and Table 5. Again, the best results are obtained by the DEC algorithm and two different linkages are provided with a low SE value (i.e. 1.1039 mm² and 1.1362 mm²). The best SE values provided by the DEE and DE algorithms are 1.5335 mm² and 1.7308 mm², respectively. In this example, the differences between the three algorithms are significant. The worst result in accuracy is for the DE algorithm, obtaining a linkage with dimensions that are not similar to the dimensions given by DEC and DEE.

The best results obtained for accuracy and efficiency are shown in Fig. 13. From the point of view of accuracy (see Fig. 13a, c and e), results show again that the DEE algorithm works well when the number of generations and size of the population is low. However, the trend of this behavior changes when these parameters grow. In this case, the best results in accuracy are obtained for the DEC algorithm. The efficiencies of the three algorithms are shown in Fig. 13b, d and f. In this case, the best results again demonstrate the better efficiency of the DE algorithm. The comparison of DEE and DEC shows the same behavior as in the previous example, i.e. below 200 individuals, the best efficiency is

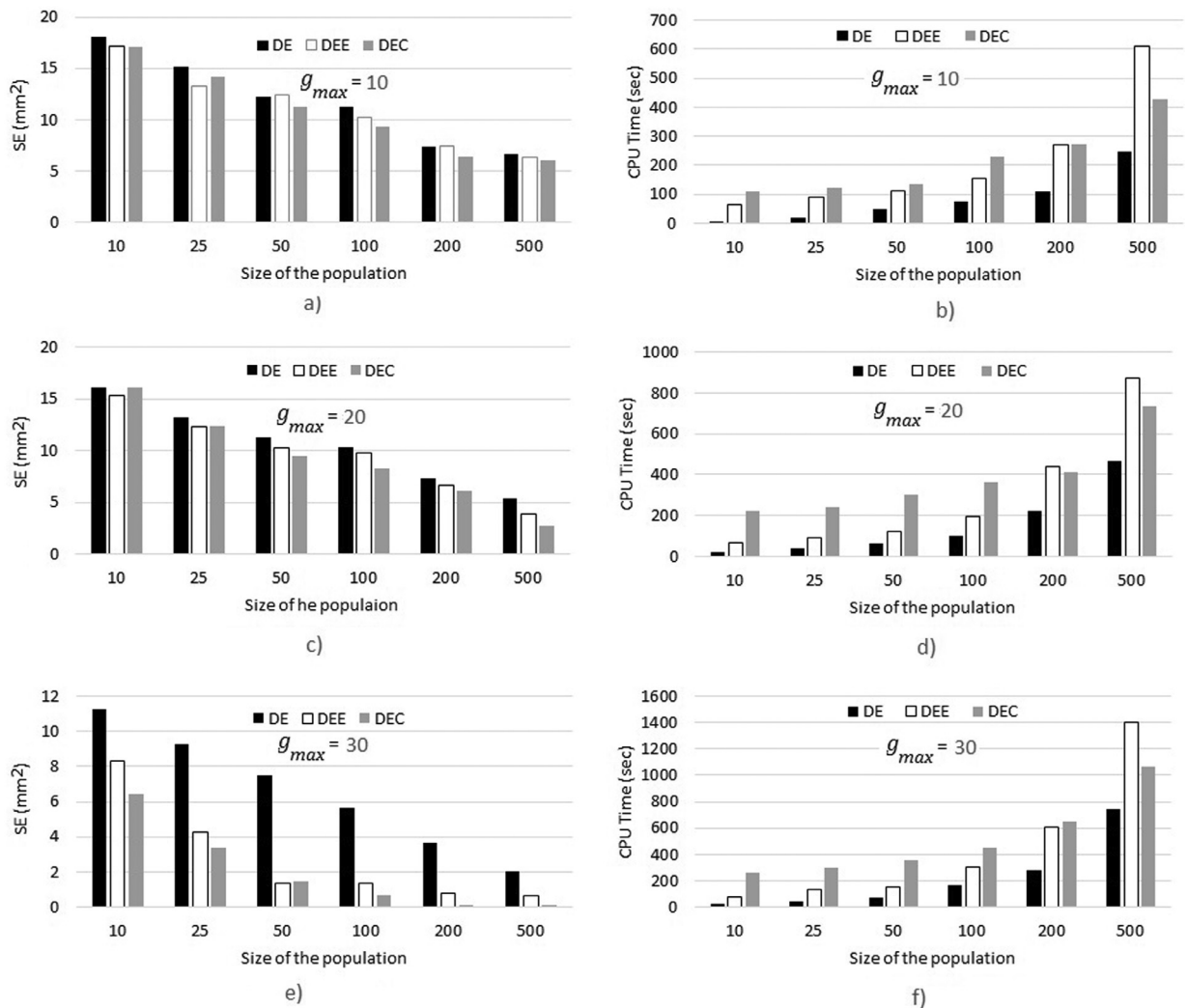


Fig. 13. Synthesis Error and CPU Time obtained for the closed path with different number of individuals.

Table 6

Number of successes in which an accuracy of 80% is exceeded.

g_{max}	Algorithm	Size of the population, s_{max}					
		10	25	50	100	200	500
10	DE	0	0	1	1	3	7
	DEE	0	1	1	1	3	6
	DEC	0	0	2	5	6	8
20	DE	0	0	0	3	7	7
	DEE	1	3	1	3	8	10
	DEC	0	3	3	9	9	10
30	DE	1	1	3	4	7	9
	DEE	1	2	2	4	6	10
	DEC	2	5	8	7	9	10

for the DEE algorithm, however, with 200 individuals or more, the trend changes and the best efficiency is for the DEC algorithm.

Table 6 shows the robustness obtained in this example. The robustness of the DEC algorithm is greater in all cases. Nevertheless, with a population of 500 individuals, complete success is obtained with the DEE and DEC algorithms.

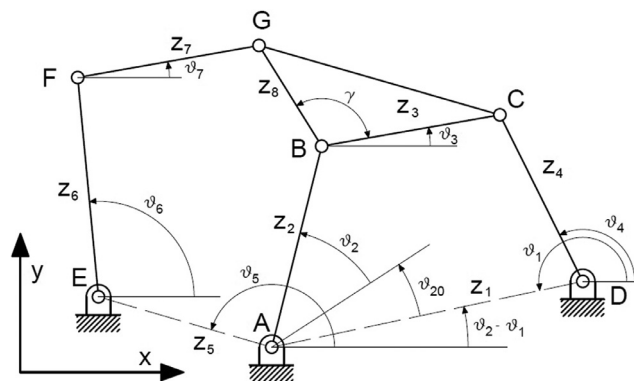


Fig. 14. Scheme of a Stephenson III linkage and coordinates used in the algorithm.

Table 7

Precision points used in the desired function
(all units in deg).

No	ϑ_{20}	ϑ_6
1	0	0
2	12.4	1.1
3	24.8	4.2
4	37.2	8.7
5	49.6	13.9
6	62.0	18.7
7	74.4	22.2
8	86.8	24
9	99.3	24
10	111.7	24
11	124.1	24
12	136.5	24
13	148.9	24
14	161.3	24
15	173.7	24
16	186.2	24
17	198.6	24
18	211.0	24
19	223.4	24
20	235.8	24
21	248.2	24
22	260.6	24
23	273.1	24
24	285.5	22.2
25	297.9	18.7
26	310.3	13.9
27	322.7	8.7
28	335.1	4.2
29	347.5	1.1
30	360.0	0

5.3. Case study 3: function generation with a six-bar linkage and a single dwell

In this example, the synthesis of a function generation linkage is considered. To this end, the Stephenson III mechanism, whose kinematic scheme is depicted in Fig. 14, is used. The desired function is a single dwell defined by means of 30 precision points (see Table 7). The goal is to obtain coordination between link 2 (input link) and link 6 (output). The Grashof condition should be fulfilled in the results.

The best linkages obtained after the optimization are shown in Fig. 15. Table 8 shows the SE values and the design variables obtained by the three algorithms. The best linkage has been obtained with the DEC algorithm and it is shown in Fig. 15a. The desired and generated functions for this linkage are shown in Fig. 15b. It is clear that there is a good match between both curves. The error (see Table 8) is $SE = 0.0048 \text{ rad}^2$. The results obtained by the DEE algorithm are shown in Fig. 15c and d. The former shows the scheme of the linkage and the latter the generated and desired curves. In this case, there is an appreciable difference between the two curves. Fig. 15e and f show the best results for the DE algorithm. The differences in this case are significant.

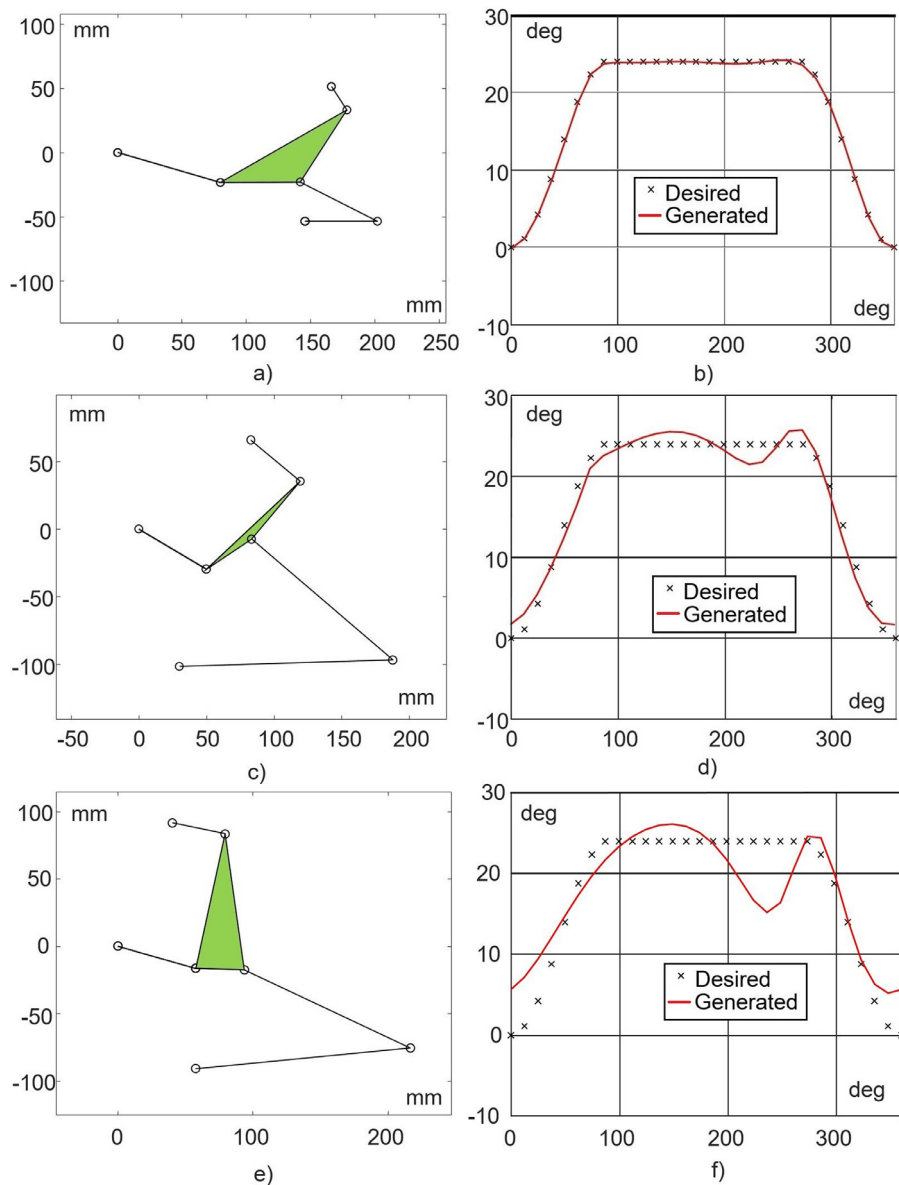


Fig. 15. Results obtained in function generation problem: a) best linkage obtained with DEC and b) the comparison of desired and generated function. In c) linkage obtained with DEE and, d) their generated and desired functions. In e) best solution obtained with DE and f) the comparison of generated desired functions.

Table 8

Best results obtained in the Stephenson III linkage.

Algorithm	SE (rad ²)	Variables											
		ϑ_1 (deg)	ϑ_5 (deg)	ϑ_{20} (deg)	z_1 (mm)	z_2 (mm)	z_3 (mm)	z_4 (mm)	z_5 (mm)	z_6 (mm)	z_7 (mm)	z_8 (mm)	γ (deg)
DEC	0.0048	17.1	−20.2	−56.9	174.1	21.7	113.3	83.3	155.3	56.3	67.2	62.1	−330.4
DEE	0.6442	46.2	−70.8	−29.8	102.2	45.0	98.9	59.8	106.2	158.4	137.1	39.2	−339.2
DE	2.0058	66.1	−57.7	−11.7	99.9	39.9	102.0	60.0	107.6	159.9	136.0	36.0	−280.8

In the same way as was done in the previous examples, Fig. 16 shows the comparative results among the three algorithms in terms of accuracy and efficiency. In general, the behaviors of the three algorithms are similar to those results found in path generation problems. However, in this case, there is a remarkable difference between the maximum accuracy obtained with each algorithm. In fact, the DEC algorithm achieves a much greater precision than the other two. It is remarkable that with a low population and a reduced maximum number of generations, the errors obtained are similar. This effect was

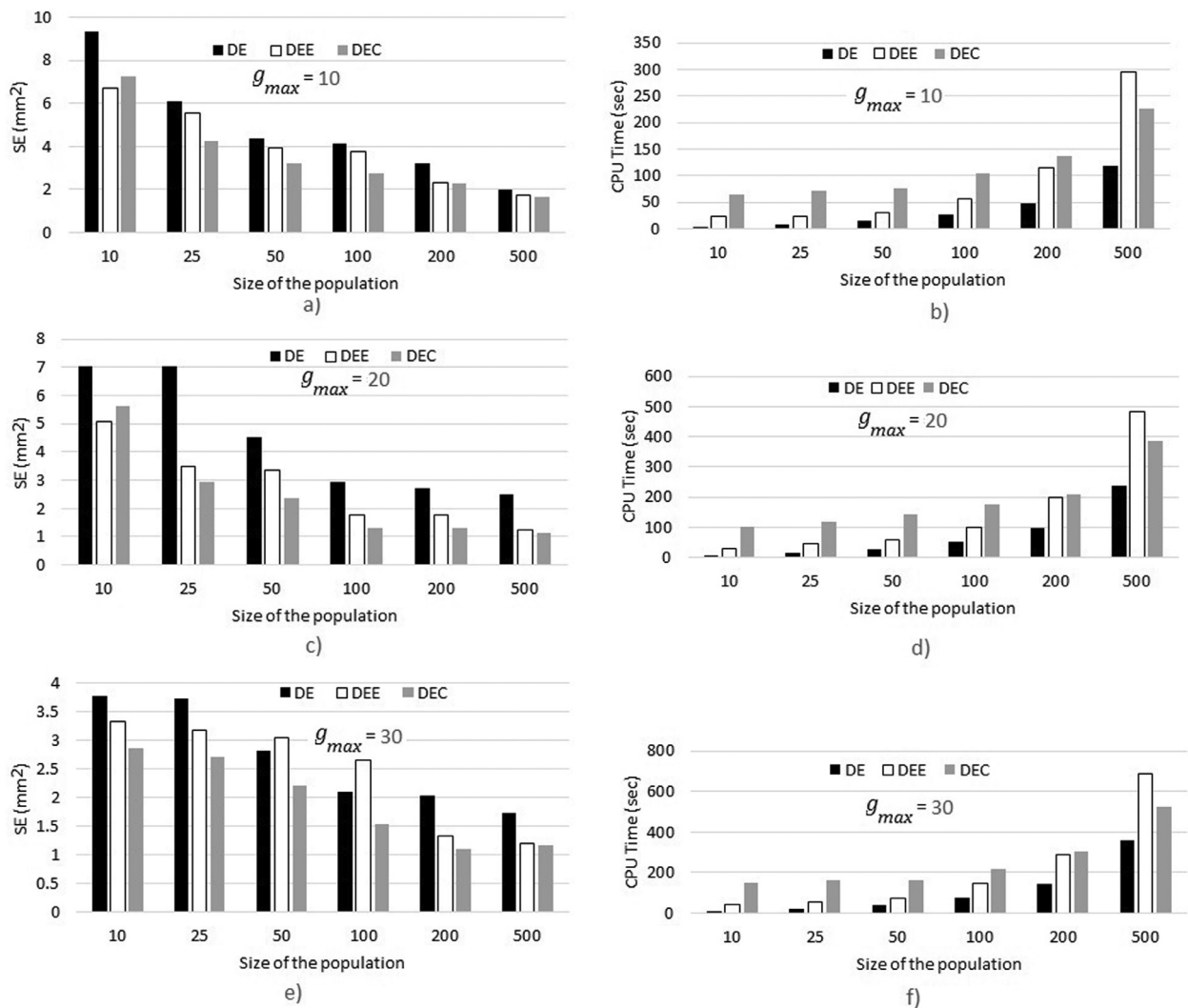


Fig. 16. Accuracy and CPU time of the results obtained with different populations and generations in the synthesis of function generation.

Table 9

Number of successes in the function generation problem where an accuracy of 80% is exceeded.

g_{max}	Algorithm	Size of the population, s_{max}					
		10	25	50	100	200	500
10	DE	0	0	0	1	3	5
	DEE	0	1	1	2	4	6
	DEC	0	0	1	3	5	7
20	DE	0	0	0	1	5	8
	DEE	0	1	1	3	7	9
	DEC	0	1	3	7	8	10
30	DE	0	1	2	3	7	8
	DEE	1	2	3	6	9	10
	DEC	0	1	4	8	10	10

not observed in the previous examples. Table 9 shows the results obtained for robustness. The trend is quite similar to those obtained in previous examples. As can be observed, when the size of population is 500 and the maximum number of generation is 30, the robustness in DEE and DEC is similar. In other cases, robustness is lower in DEE and DE algorithms than in DEC.

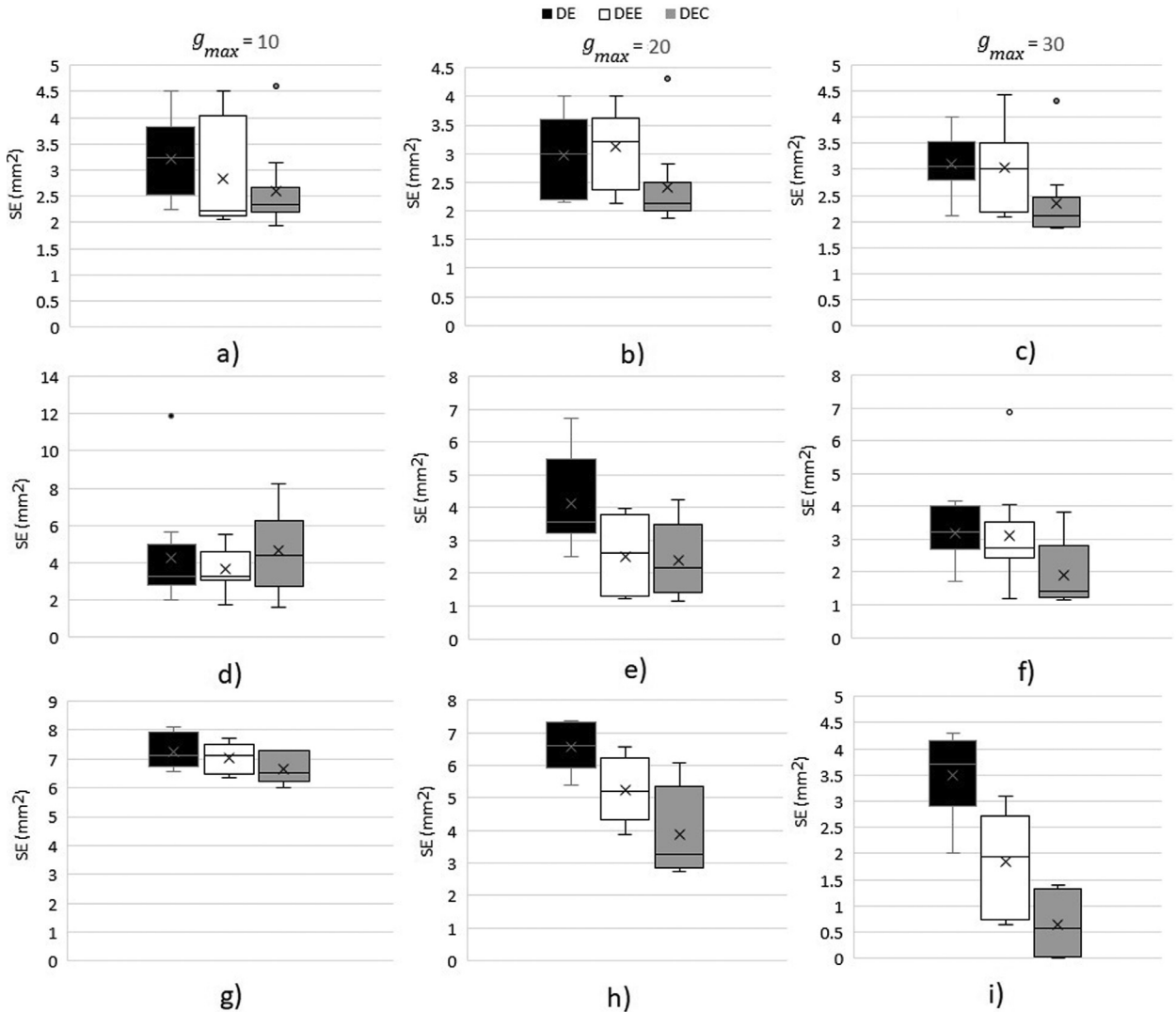


Fig. 17. Statistical results for accuracy with 500 individuals in the three examples presented.

5.4. Statistical analysis of accuracy and efficiency

In this subsection, the statistical comparison of the three methods is discussed. To this end, one-way analysis of variance (ANOVA) was performed on the results obtained in the previous examples when the number of individuals is 500. Significant values are considered when $p < 0.05$. Statistical graphical results of the accuracy are shown in Fig. 17, where the graphs in each row belong to the same example and each column provides the results for different numbers of maximum generations. Thus, the results in Fig. 17a–c show the statistical results for accuracy in case study 1. Although the minimum error was obtained with the DEC algorithm, in all cases, the differences were not significant ($p = 0.2836$) with 10 generations. With 20 generations, the difference between the algorithms were not strictly significant ($p = 0.0769$), but they were close to being significant. However, with 30 generations, the differences were clearly significant with $p = 0.0398$.

In the second case study the graphical values of the statistical analysis are shown in Fig. 17d–f. Again, the differences cannot be considered significant with 10 generations ($p = 0.623$). Nevertheless, these differences are significant with 20 generations ($p = 0.0068$) and 30 generations ($p = 0.037$). Something similar happens in the third case study (see Fig. 17g–i). In this case there are clear differences when the graphs are observed. It seems that the DEC algorithm is the best in all cases. However, the statistical analysis shows that they cannot be considered significant with 10 generations ($p = 0.2032$). Significant differences are clear with 20 generations ($p = 0.0025$) and 30 generations ($p = 0.0001$). In the same way, statistical

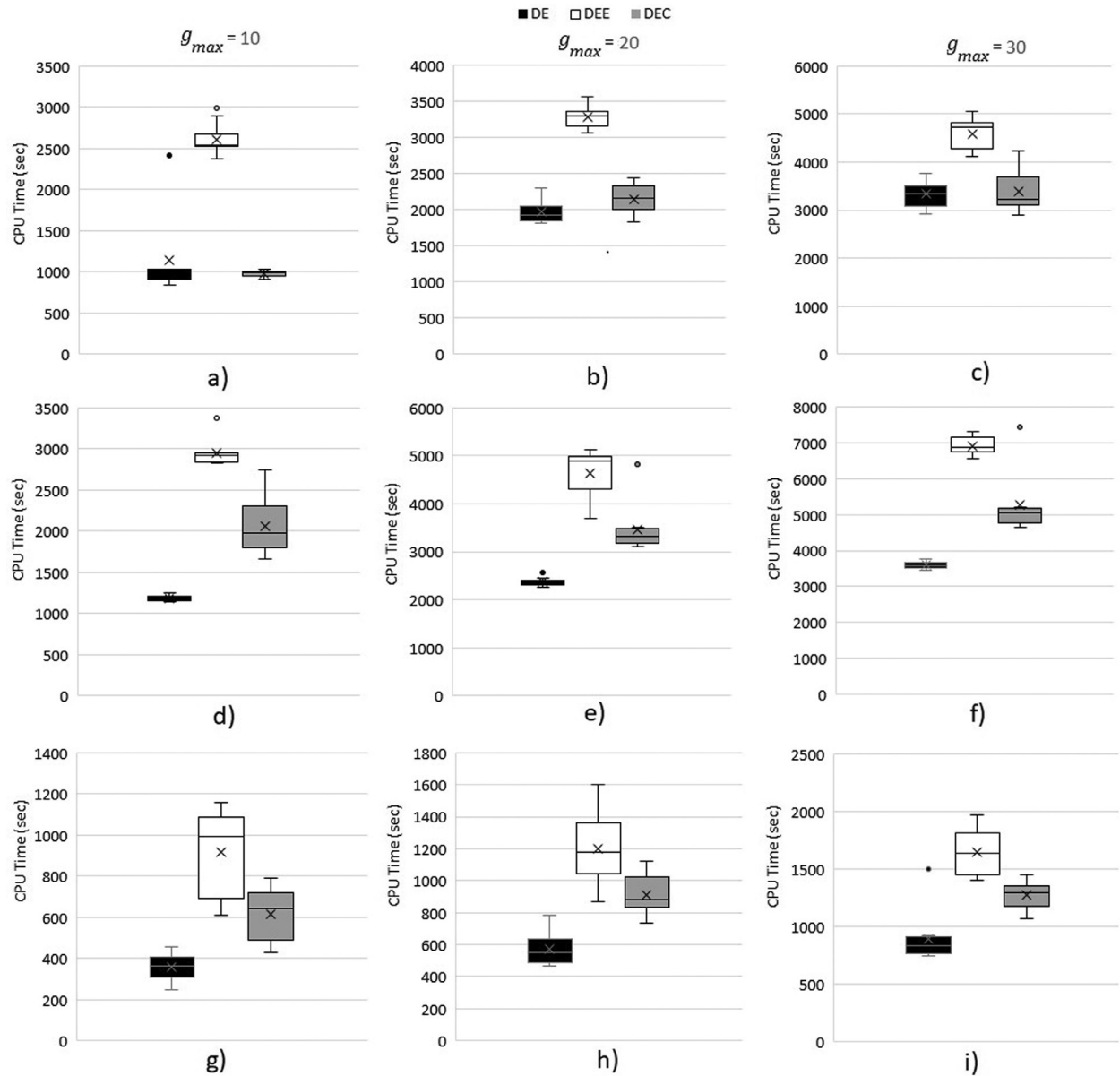


Fig. 18. Statistical results for efficiency with 500 individual in the three examples presented.

graphical results are shown for CPU time values in Fig. 18. In this case differences were strongly significant in all cases (i.e. $p < 0.001$).

6. Conclusions

In this paper the effect of hybridizing local search in a DE algorithm has been presented. The goal is to improve the capabilities of exploration of DE with the exploitation advantages of local search. To this end, three algorithms were developed and their results were compared. To improve the efficiency, two strategies were considered. The first one is that the algorithms incorporate novel approaches for the comparison of the targets (i.e. FECMI and CERED). The second one is a strategy to select a set of individuals to be optimized locally. The Elite-based strategy (DEE) selects only the best individuals to be optimized whereas the Cluster-based approach (DEC) divides the individuals into groups with similar characteristics and selects the best one in each group for optimization.

The performance of the three algorithms was evaluated using three case studies, namely two examples of path generation with a four-bar linkage, and one example of a function generator with a Stephenson III linkage. The test implies running each

case study ten times in order to obtain statistical relevance. The results show that in no case was the efficiency improved by hybrid approaches. In fact, the DE algorithm always achieves the best time to convergence. This means that hybridization involves a higher computational cost due to the necessity of carrying out local search. The maximum accuracy is obtained with 30 generations and 200 and 500 individuals. With these results, the DEC algorithm achieves greater efficiency than DEE. The greatest accuracy is obtained by the DEC algorithm, although, sometimes the differences are not significant. The examples also show that robustness is highest in DEC, achieving accurate results (more than 80% of the maximum accuracy) in most of the tests carried out. DEE achieves slightly lower values of robustness and DE provides the lowest values.

From the results obtained in this paper, it can be concluded that hybridization improves the accuracy of the results because it enables better exploration of the space of the design variables. Cluster-based hybridization allows the selection of the best individuals to be optimized by means of local search. Thus, from the point of view of accuracy, it is the best approach. However, due to high number of factors and variables that influence the results, future work is needed and other strategies should be investigated.

Acknowledgment

This work was developed in the framework of the Project [DPI2010-18316](#) funded by the Spanish Ministry of Economy and Competitiveness.

References

- [1] G.N. Sandor, A.G. Erdman, *Advanced Mechanism Design: Analysis and Synthesis*, Prentice-Hall, New Jersey, 1984.
- [2] R.I. Alizade, I.G. Novruzbebekov, G.N. Sandor, Optimization of 4-bar function generating mechanisms using penalty functions with inequality and equality constraints, *Mech. Mach. Theory* 10 (1975) 327–336. [https://doi.org/10.1016/0094-114X\(75\)90077-4](https://doi.org/10.1016/0094-114X(75)90077-4).
- [3] H. Zhou, Dimensional synthesis of adjustable path generation linkages using the optimal slider adjustment, *Mech. Mach. Theory* 44 (2009) 1866–1876. <https://doi.org/10.1016/j.mechmachtheory.2009.03.010>.
- [4] A. Kunjur, S. Krishnamurthy, A robust multi-criteria optimization approach, *Mech. Mach. Theory* 32 (1997) 797–810. [https://doi.org/10.1016/S0094-114X\(97\)00007-4](https://doi.org/10.1016/S0094-114X(97)00007-4).
- [5] A.D. Belegundu, T.R. Chandrupatla, *Optimization Concepts and Applications in Engineering*, Cambridge University Press, USA, 2011.
- [6] S.N. Kramer, G.N. Sandor, Selective precision synthesis. A general method of optimization for planar mechanism, *J. Eng. Ind.* 2 (1975) 678–701, doi:10.1115/1.3438634.
- [7] V.N. Sohoni, E.J. Haug, A state space technique for optimal design of mechanism, *ASME J. Mech. Des.* 104 (4) (1982) 792–798, doi:10.1115/1.3256438.
- [8] G.H. Sutherland, J.N. Siddal, Dimensional synthesis of linkages by multifactor optimization, *Mech. Mach. Theory* 9 (1974) 81–95. [https://doi.org/10.1016/0094-114X\(74\)90009-3](https://doi.org/10.1016/0094-114X(74)90009-3).
- [9] X. Li, S. Wei, Q. Liao, Y. Zhang, A novel analytical method for function generation synthesis of planar four-bar linkages, *Mech. Mach. Theory* 101 (2016) 222–235. <https://doi.org/10.1016/j.mechmachtheory.2016.03.013>.
- [10] R. Aviles, J. Vallejo, I.F. de Bustos, J. Aguirrebeitia, G. Ajuria, Optimum synthesis of planar linkages using a strain-energy error function under geometric constraints, *Mech. Mach. Theory* 45 (2010) 65–79. <https://doi.org/10.1016/j.mechmachtheory.2009.08.002>.
- [11] F.T. Sanchez-Marin, A. Perez-Gonzalez, Open-path synthesis of linkages through geometrical adaptation, *Mech. Mach. Theory* 39 (2004) 943–955. <https://doi.org/10.1016/j.mechmachtheory.2004.04.002>.
- [12] J. Mariappan, S. Krishnamurthy, A generalized exact gradients method for mechanism synthesis, *Mech. Mach. Theory* 31 (1996) 413–421. [https://doi.org/10.1016/0094-114X\(95\)00077-C](https://doi.org/10.1016/0094-114X(95)00077-C).
- [13] R. Sancibrian, E.G. Sarabia, A. Sedano, J.M. Blanco, A general method for the optimal synthesis of mechanisms using prescribed instant center positions, *Appl. Math. Model.* 40 (2016) 2206–2222. <https://doi.org/10.1016/j.apm.2015.09.032>.
- [14] R. Sancibrian, Improved GRG method for the optimal synthesis of linkages in function generation problems, *Mech. Mach. Theory* 46 (2011) 1350–1375. <https://doi.org/10.1016/j.mechmachtheory.2011.05.011>.
- [15] A. Sedano, R. Sancibrian, A. de Juan, F. Viadero, F. Egaña, Hybrid optimization approach for the design of mechanism using a new error estimator, *Math. Probl. Eng.* 3 (2012) 1–20. <http://dx.doi.org/10.1155/2012/151590>.
- [16] I. Ullah, S. Kota, Optimal synthesis of mechanism for path generation using fourier descriptor and global search methods, *ASME J. Mech. Des.* 119 (4) (1997) 504–510, doi:10.1115/1.2826396.
- [17] G. Galán, F.J. Alonso, J.M. del Castillo, Shape optimization for path synthesis of crank-rocker using wavelet-based neural network, *Mech. Mach. Theory* 44 (2009) 1132–1143. <https://doi.org/10.1016/j.mechmachtheory.2008.09.006>.
- [18] F. Nadal, J.A. Cabrera, A. Ballester, J.J. Castillo, Turning functions in optimal synthesis of mechanisms, *J. Mech. Des.* 137 (2015) 062302–1–10, doi:10.1115/1.4029825.
- [19] J.H. Holland, *Adaptation in Natural and Artificial System*, The University of Michigan Press, Michigan, 1975.
- [20] G.D. Roston, R.H. Sturges, Genetic algorithm synthesis of four-bar mechanism, *Artif. Intell. Eng. Des. Anal. Manuf.* 10 (1996) 371–390, doi:10.1017/S0890060400001700.
- [21] A. Kunjur, S. Krishnamurthy, Genetic algorithms in mechanism synthesis, *J. Appl. Mech. Robot.* 4 (1997) 18–24.
- [22] J.A. Cabrera, A. Simon, M. Prado, Optimal synthesis of mechanisms with genetic algorithm, *Mech. Mach. Theory* 37 (2002). 1165–1117. [https://doi.org/10.1016/S0094-114X\(02\)00051-4](https://doi.org/10.1016/S0094-114X(02)00051-4).
- [23] A. Smaili, N. Diab, A new error approach to shape optimization for closed path synthesis of planar mechanism, *ASME J. Mech. Des.* 129 (2007) 941–948, doi:10.1115/1.2753164.
- [24] A. Sardashi, H.M. Daniali, S.M. Varedi, Optimal free-defect synthesis of four-bar linkages with joint clearance using PSO algorithm, *Meccanica* 48 (2013) 1681–1693. <https://doi.org/10.1007/s11012-013-9699-6>.
- [25] R. Storn, K. Price, Differential Evolution. A simple and efficient heuristic scheme for global optimization over continuous spaces, *J. Glob. Optim.* 11 (1997) 341–359.
- [26] P.S. Shiakolas, D. Koladiya, J. Kebrle, On the optimum synthesis of six-bar linkages using differential evolution and the geometric centroid of precision technique, *Mech. Mach. Theory* 40 (2005) 319–335. <https://doi.org/10.1016/j.mechmachtheory.2004.07.005>.
- [27] J.A. Cabrera, F. Nadal, J.P. Muñoz, A. Simon, Multiobjective constrained optimal synthesis of planar mechanisms using a new evolutionary algorithm, *Mech. Mach. Theory* 42 (2007) 791–806. <https://doi.org/10.1016/j.mechmachtheory.2006.07.004>.
- [28] R.R. Bulatovic, S.R. Dordevic, On the optimum synthesis of a four-bar linkage using differential evolution and method of variable controlled deviations, *Mech. Mach. Theory* 44 (2009) 235–246. <https://doi.org/10.1016/j.mechmachtheory.2008.02.001>.
- [29] F. Peñuñuri, R. Peon-Escalante, C. Villanueva, D. Pech-Oy, Synthesis of mechanisms for single hybrid task using differential evolution, *Mech. Mach. Theory* 47 (2011) 1335–1349. <https://doi.org/10.1016/j.mechmachtheory.2011.05.013>.

- [30] S.H. Kafash, A. Nahvi, Optimal synthesis of four-bar path generator linkages using circular proximity function, *Mech. Mach. Theory* 115 (2017) 18–34. <https://doi.org/10.1016/j.mechmachtheory.2017.04.010>.
- [31] A. Ortiz, J.A. Cabrera, F. Nadal, A. Bonilla, Dimensional synthesis of mechanism using differential evolution with auto-adaptive control parameters, *Mech. Mach. Theory* 64 (2013) 210–229. <https://doi.org/10.1016/j.mechmachtheory.2013.02.002>.
- [32] T.W. Liao, Two hybrid differential evolution algorithms for engineering design optimization, *Appl. Soft Comput.* 10 (2010) 1188–1199. <https://doi.org/10.1016/j.asoc.2010.05.007>.
- [33] M.A. Ahandani, M.T. Vakil-Baghmisheh, M. Takebi, Hybridizing local search algorithms for global optimization, *Comput. Optim. Appl.* 59 (2014) 725–748. <https://doi.org/10.1007/s10589-014-9652-1>.
- [34] K. Zhang, Q. Huang, Y. Zhang, J. Song, J. Shi, Hybrid lagrange interpolation differential evolution algorithm for path synthesis, *Mech. Mach. Theory* 134 (2019) 512–540. <https://doi.org/10.1016/j.mechmachtheory.2019.01.012>.
- [35] E.R. Hurschka, R.J.G.B. Campello, A.A. Freitas, A.C.P.L.F. De Carballo, A survey of evolutionary algorithms for clustering, *IEEE Trans. Syst. Man Cybern.* 99 (2009) 133–155, doi:10.1109/TSMCC.2008.2007252.
- [36] A. Abraham, S. Das, S. Roy, in: *Swarm Intelligence Algorithms for Data clustering, Soft Computing for Knowledge Discovery and Data Mining, Part IV*, Springer, 2007, pp. 279–313.
- [37] A.C. Martinez-Estudillo, C. Hervás-Martínez, F.J. Martinez-Estudillo, N. García-Pedrajas, Hybridization of evolutionary algorithms and local search by means of clustering method, *IEEE Trans. Syst. Man Cybern. Part B (Cybern.)* 36 (2006) 534–545, doi:10.1109/TSMCB.2005.860138.
- [38] M. Friedlander, M. Schmidt, Hybrid deterministic-stochastic methods for data fitting, *SIAM J. Sci. Comput.* 35 (2012) A1380–A1405. <https://doi.org/10.1137/110830629>.
- [39] N. Noman, H. Iba, Accelerating differential evolution using an adaptive local search, *IEEE Trans. Evol. Comput.* 12 (2008) 107–125, doi:10.1109/TEVC.2007.895272.
- [40] S. Lloyd, Least squares quantization in PCM, *IEEE Trans. Inf. Theory* 28 (1982) 129–137.34.
- [41] P. Hajela, J. Lee, Constrained genetic search via schema adaptation, an immune network solution, *Struct. Optim.* 12 (1996) 11–15. <https://doi.org/10.1007/BF01270439>.
- [42] A. Homaifar, S.H.Y. Lai, X. Qi, Constrained optimization via genetic algorithms, *Simulation* 62 (1994) 242–254. <https://doi.org/10.1177/003754979406200405>.